



Kierunek: Informatyka

2024/2025

Kamil Rataj

PRACA INŻYNIERSKA

**System rozpoznawania oraz tłumaczenia alfabetu
migowego z wykorzystaniem sztucznej inteligencji**

Promotor pracy:

dr inż. Daniel Król, prof. AT

Tarnów, 2025 r.

Spis treści

1. Wprowadzenie.....	5
1.1. Język migowy	6
1.1.1. Alfabet palcowy (daktylografia).....	6
1.1.2. Amerykański Język Migowy (<i>ASL</i>).....	6
1.1.3. Międzynarodowy Alfabet Palcowy.....	7
1.2. Wykluczenie osób niesłyszących.....	9
1.3. Cyfrowe rozpoznawanie obrazu.....	9
1.4. Uczenie maszynowe.....	10
1.4.1. Terminologia uczenia maszynowego.....	10
1.5. Splotowe(konwolucyjne) sieci neuronowe.....	11
1.6. Uczenie transferowe.....	11
2. Wykorzystane narzędzia i technologie	12
2.1. Język programowania i środowiska programistyczne	12
2.1.1. Język Python 3.....	12
2.1.2. Platforma Google Colab	13
2.1.3. JetBrains Pycharm Professional IDE.....	13
2.1.4. Notatnik Jupyter.....	14
2.1.5. Qt Framework.....	14
2.2. Technologie sztucznej inteligencji i cyfrowego przetwarzania obrazu.....	15
2.2.1. Pakiet MediaPipe Solutions.....	15
2.2.2. Biblioteka OpenCV.....	16
2.2.3. XNNPACK.....	17
2.2.4. Biblioteka TensorFlow.....	18
2.2.5. Platforma Kaggle.....	18
2.3. System kontroli wersji i zarządzania projektem.....	19
2.3.1. System Git.....	19
2.3.2. Platforma GitHub.....	19
2.4. Interfejs użytkownika i środowisko wizualne.....	21
2.4.1. Qt for Python.....	21
2.4.2. Qt Designer.....	22
2.4.3. QDarkStyle.....	22
2.5. Moduł syntezy mowy.....	23
2.5.1. Biblioteka pyttsx3.....	23
2.5.2. Silnik(engine) syntezy mowy.....	23
3. Trening modelu klasyfikacyjnego.....	24
3.1. Środowisko.....	24
3.2. Zbiór danych treningowych.....	24
3.2.1. Źródło.....	25
3.2.2. ETL zbioru danych.....	25
3.3. MediaPipe Model Maker.....	26
3.4. Podział zbioru danych treningowych.....	27
3.5. Parametryzacja.....	28
3.5.1. Hiperparametry modelu.....	28
3.5.2. Hiperparametry treningu.....	29
3.5.3. Zastosowane hiperparametry.....	29
3.6. Ewaluacja modelu.....	30

3.7. Eksport modelu.....	30
3.8. Podsumowanie.....	30
3.9. Wykres przebiegu procesu treningu.....	32
3.9.1. Wykres dokładności (ang. <i>accuracy</i>).....	32
3.9.2. Wykres funkcji straty (ang. <i>loss</i>).....	33
3.9.3. Wykres współczynnika uczenia(ang. <i>learning rate</i>).....	34
4. Implementacja systemu.....	35
4.1. Środowisko uruchomieniowe.....	35
4.1.1. Wirtualne środowisko Python.....	35
4.1.2. Aplikacja wykonywalna (.exe).....	36
4.1.3. Urządzenie rejestrujące.....	36
4.1.4. Biblioteka TensorFlow Lite.....	36
4.2. Przechwytywanie obrazu.....	37
4.3. Analiza obrazu.....	38
4.3.1. Wykrywanie dłoni.....	39
4.3.2. Ustalanie punktów orientacyjnych i śledzenie.....	40
4.4. Klasyfikacja gestów.....	41
4.4.1. Model osadzeń (ang. <i>embedding</i>).....	41
4.4.2. Model klasyfikacyjny.....	41
4.5. Przygotowanie wyników.....	42
4.5.1. Wizualizacja punktów orientacyjnych dłoni.....	43
4.5.2. Przygotowanie i formatowanie wyniku.....	44
4.6. Interfejs użytkownika	45
4.6.1. Podgląd obrazu.....	46
4.6.2. Wyniki.....	46
4.6.3. Ustawienia.....	47
4.6.4. Informacje o autorze.....	50
4.6.5. Konsola diagnostyczna.....	51
5. Testy systemu.....	52
5.1. Ewaluacja modelu.....	52
5.2. Testy urządzeń rejestrujących.....	53
5.3. Testy rozdzielczości kamery.....	55
5.4. Testy interfejsów kamery.....	57
5.5. Testy funkcjonalne.....	58
5.5.1. Test rozpoznawania gestów.....	59
5.5.2. Test generowania tekstu i mowy.....	60
5.5.3. Test interfejsu użytkownika.....	61
6. Podsumowanie i wnioski.....	62

1. Wprowadzenie

W dobie sztucznej inteligencji coraz częściej szukamy rozwiązań aktualnych problemów korzystając z jej wszechstronnych technik. Wśród nas jest wiele osób niepełnosprawnych, którzy mierzą się na co dzień z wykluczeniami społecznymi jak i cyfrowymi. Niniejsza praca skupia się na przypadku osób niesłyszących i słabosłyszących, które używają języka migowego do porozumiewania się oraz jest ich podstawowym środkiem komunikacji. Tworzy to wiele barier oraz ograniczeń w ich życiu prywatnym jak i zawodowym [1]. Rynek nie oferuje wielu rozwiązań tych problemów oraz rozwiązania nie są kompleksowe, pomijają istotne wyzwanie jakim jest komunikacja osób niesłyszących lub niewidomych. Niniejsza praca jest próbą rozwiązania tych problemów, ponadto prezentuje możliwości zastosowań technik sztucznej inteligencji, a w szczególności modeli powstałych w ramach *MediaPipe Google* do przetwarzania obrazu i rozpoznawania statycznych gestów dłoni.

Celem niniejszej pracy była implementacja oprogramowania wspomagającego komunikację osób niesłyszących i słabosłyszących. System opiera się na zaawansowanych technikach wizji komputerowej oraz technologii sztucznej inteligencji. Zadaniem systemu jest rozpoznawanie statycznych znaków języka migowego z kamery i tłumaczenie ich na inne formy komunikacji takie jak tekst pisany czy mowa syntetyzowana czasie rzeczywistym. System skupia się na literach Amerykańskiego alfabetu palcowego. Aplikacja przeznaczona jest dla osób posługujących się językiem angielskim, szczególnie Amerykanów.

System został zaprojektowany z myślą o prostocie użytkowania oraz efektywności rozpoznawania gestów. Wykorzystanie modeli sztucznej inteligencji umożliwia szybkie przetwarzanie obrazu i interpretację znaków w czasie rzeczywistym, co zwiększa dostępność komunikacji dla osób niesłyszących. Implementacja uwzględnia optymalizację algorytmów w celu zapewnienia płynności działania nawet na urządzeniach o ograniczonej mocy obliczeniowej. Dodatkowo, system może stanowić podstawę do dalszego rozwoju, np. integracji z urządzeniami mobilnymi, aplikacjami internetowymi czy asystentami głosowymi, co otwiera nowe możliwości dla użytkowników.

1.1. Język migowy

Język migowy to wizualno-przestrzenny system komunikacji dla osób niesłyszących i słabosłyszących wykorzystujący kanał wzrokowy, jest on językiem naturalnym. Wykorzystuje on układy palców i dłoni, mimikę twarzy, postawę ciała oraz pozycję w przestrzeni [2]. Nie jest uniwersalny, ponieważ na świecie istnieje wiele (około 200 - 300) odmiennych języków migowych znacznie różniących się od siebie, występują także regionalizmy. Ponad 5 % populacji, czyli około 430 milionów osób zmaga się z upośledzającą utratą słuchu. Według szacunków liczba ta zwiększy się do około 700 milionów osób, do 2050 roku [3].

1.1.1. Alfabet palcowy (daktylografia)

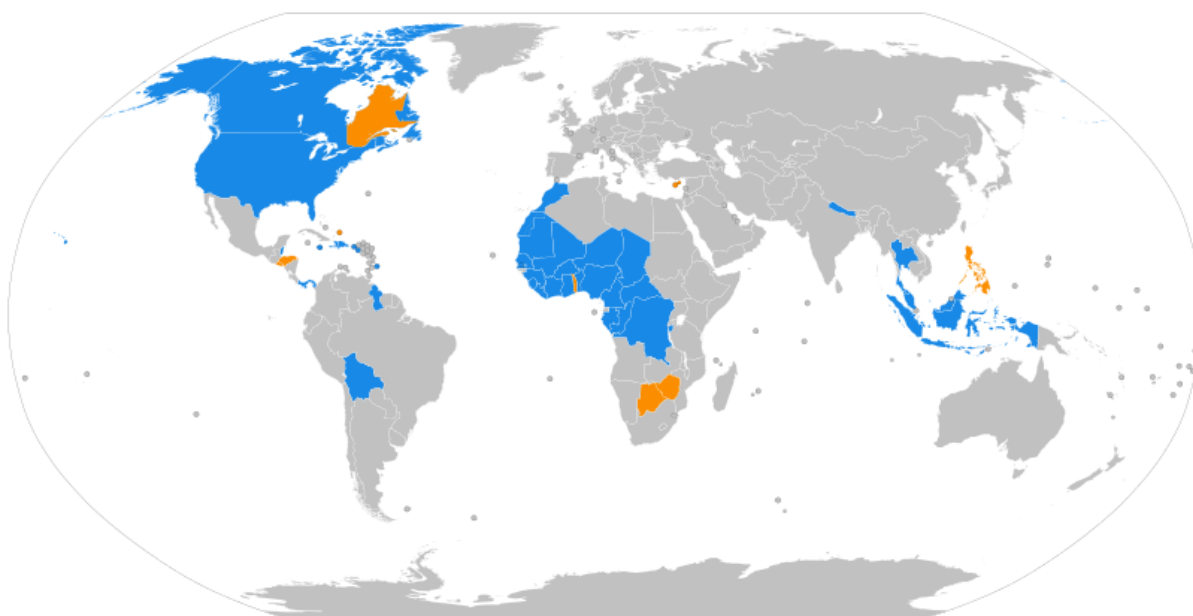
Alfabet palcowy to ułożony w tradycyjnym porządku unikalny zbiór znaków migowych (daktylogramów) odpowiadających poszczególnym literom, zazwyczaj reprezentowanych określonym gestem dłoni, ułożeniem palców i ewentualnie ruchem. Najczęściej prezentowane są za pomocą odpowiednich układów prawej ręki na wysokości twarzy po prawej stronie [4]. Są one częścią języka migowego, lecz nie są tożsame z całym językiem, ponieważ operuje on nie tylko na literach, a również na fonemach [2]. Pierwszy pełny opis alfabetu palcowego wraz z ilustracjami pochodzi z 1562 r. Opisał go *Giovanni Battista della Porta* w książce „*De furtivis literarum notis*”.

1.1.2. Amerykański Język Migowy (ASL)

Dominującym językiem migowym w Stanach Zjednoczonych i Kanadzie (anglojęzycznej) jest *ASL* (*American Sign Language*), czyli kompletny, naturalny język, który ma takie same właściwości językowe jak język mówiony. Nie ma uniwersalnego języka migowego. W różnych krajach lub regionach używane są różne języki migowe przez co nie jest on tożsamy z Brytyjskim Językiem Migowym (*BSL*) oraz z Polskim Językiem Migowym (*PJM*), natomiast jest on jednym z najbardziej znanych i rozpowszechnionych języków migowych na całym świecie [5].

Na rysunku 1.1 przedstawiona jest mapa występowania *ASL*, na której oznaczone są obszary, gdzie:

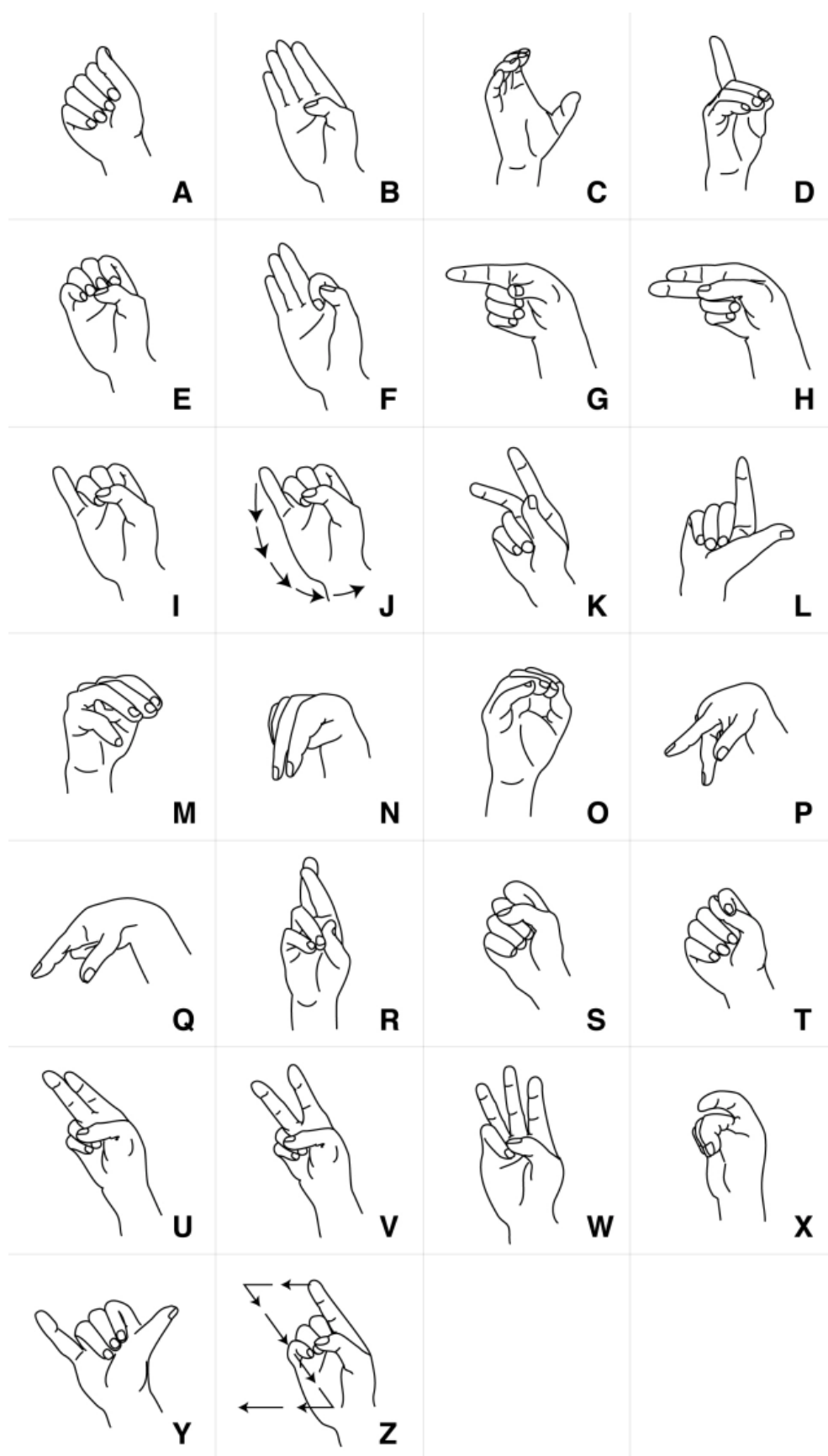
- *ASL* lub jego dialekt/pochodny jest narodowym językiem migowym(kolor niebieski)
- *ASL* jest w znacznym stopniu używany obok innego języka migowego(kolor pomarańczowy) [6]



Rysunek 1.1 Mapa występowania Amerykańskiego języka migowego. [6]

1.1.3. Międzynarodowy Alfabet Palcowy

Międzynarodowy Alfabet Palcowy (ang. *International Manual Alphabet*) to zbiór 26 znaków migowych odpowiadających poszczególnym literom alfabetu (A-Z). Jest on tożsamy z Amerykańskim Alfabetem Palcowym (*AMA*). Znaki są prezentowane za pomocą jednej dłoni, zwykle jest to dłoń dominująca. Znaki „J” oraz „Z” nie są statyczne i wymagają dodatkowych ruchów. Jest on konieczny w przypadku literowania słów oraz przy słowach i nazwach własnych, których nie obejmują dostępne w języku migi [2]. Osoba znająca zarówno język angielski oraz amerykański alfabet palcowy jest w stanie komunikować się z osobą niesłyszącą (znającą również język angielski). Na rysunku 1.2 widoczne są poszczególne znaki Amerykańskiego Alfabetu Palcowego.



Rysunek 1.2 Wykaz znaków amerykańskiego alfabetu palcowego. [7]

1.2. Wykluczenie osób niesłyszących

Osoby z uszkodzonym narządem słuchu mierzą się z wieloma ograniczeniami oraz wykluczeniami w codziennym życiu. Podstawowa przyczyna tych problemów jest oczywista, brak możliwości komunikowania się za pomocą artykułowanej dźwiękowo mowy. Oczywistość ta jednak nie jest w pełni uświadamiana osobom pełnosprawnym (słyszącym), co dodatkowo pogłębia bariery społeczne i utrudnia integrację osób niesłyszących w różnych sferach życia. Osoby słyszące mogą swobodnie korzystać z języka mówionego, telefonii, radia czy systemów komunikacji głosowej w pracy oraz edukacji. Dla osób niesłyszących takie formy komunikacji są często niedostępne lub wymagają zastosowania specjalistycznych technologii wspomagających, takich jak napisy, tłumaczenie na język migowy czy technologie rozpoznawania mowy na tekst. Niestety, ich dostępność jest ograniczona, co prowadzi do wykluczenia niesłyszących z wielu aspektów życia społecznego, zawodowego oraz edukacyjnego. Brak świadomości i dostosowania przestrzeni publicznej do potrzeb osób niesłyszących objawia się w wielu sytuacjach, takich jak brak tłumaczy języka migowego w instytucjach publicznych, ograniczony dostęp do napisów w mediach audiowizualnych czy brak odpowiednich narzędzi do komunikacji w miejscu pracy. W rezultacie osoby niesłyszące często napotykają trudności w dostępie do usług medycznych, edukacji, rynku pracy oraz życia kulturalnego i społecznego [1].

1.3. Cyfrowe rozpoznawanie obrazu

Cyfrowe lub inaczej komputerowe rozpoznawanie obrazu jest bardzo szeroką dziedziną. Zajmuje się ona przetwarzaniem danych ze statycznego obrazu, czy też np. kamery na inne reprezentacje tych danych lub na konkretne decyzje albo wnioski. Przykładowym wnioskiem jest „użytkownik znajdujący się przed kamerą prezentuje znak migowy „V””, a dane wejściowe to klatka obrazu z kamery. Zadania te na ogół wydają się proste, jest to błędne przekonanie wynikające z tego, że ludzie są istotami wzrokowymi i naturalnie oraz nieświadomie przetwarzają obraz w korze wzrokowej. Podczas rozpoznawania obrazu z użyciem komputera pojawiają się dodatkowe przeszkody takie jak szum, potrzeba kalibracji, niedoskonałość czujników (np. kamer *RGB*). Należy zwrócić szczególną uwagę na tzw. „fałszywe rozpoznania” (ang. *false-positive/false-negative*), czyli decyzje które w rzeczywistości są błędne i konieczna jest ich eliminacja [8].

1.4. Uczenie maszynowe

Uczenie maszynowe to obszar sztucznej inteligencji zajmujący się teorią oraz praktycznym zastosowaniem algorytmów do analizy danych. Jest to bardzo rozległa i interdyscyplinarna dziedzina nauki. Powstało wiele publikacji w tej dziedzinie oraz jest ona ciągle rozwijana. Pozwala na stosowanie potężnych algorytmów oraz modeli matematycznych w celu wykrywania wzorców w danych, prognozy przyszłych zdarzeń czy też podejmowania decyzji, niezależnie od człowieka. Metody opracowane dzięki uczeniu maszynowemu odgrywają coraz istotniejszą rolę w życiu codziennym oraz eliminują potrzebę zatrudniania ludzi w wielu branżach [9].

1.4.1. Terminologia uczenia maszynowego

Najważniejsze terminy stosowane w dziedzinie uczenia maszynowego:

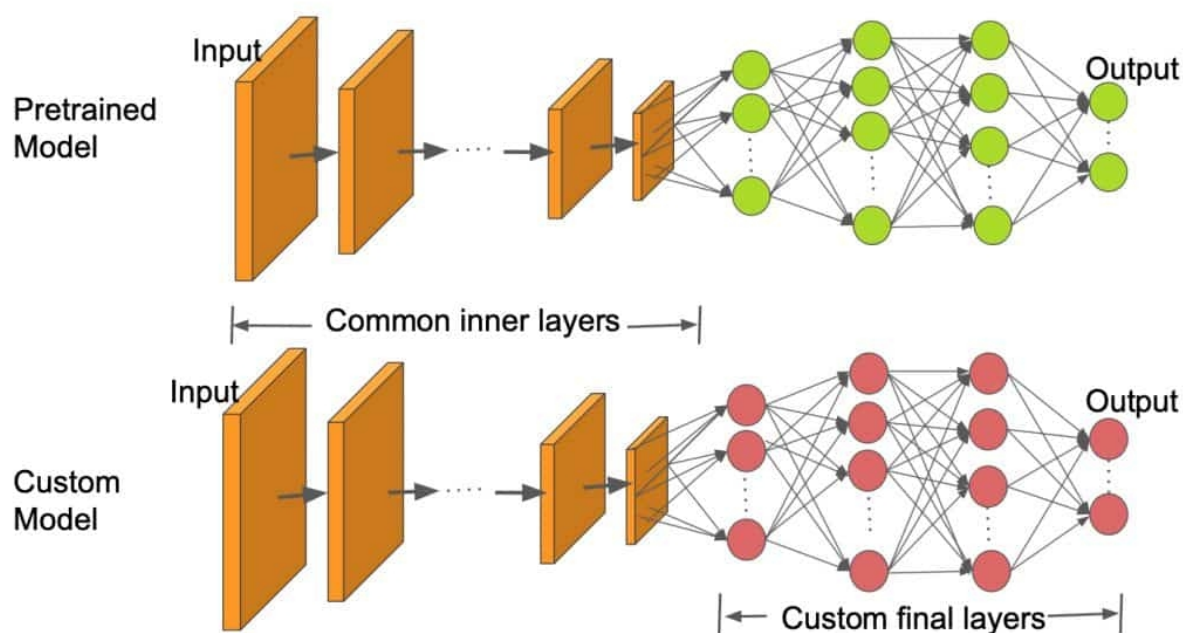
- Trenowanie/Uczenie (ang. *training*) – dopasowanie modelu, estymacja parametrów.
- Cecha (ang. *feature*) – kolumna w macierzy lub tabeli danych modelu.
- Zmienna celu (ang. *target*) – etykieta klasy, zmienna wyjściowa, wynik.
- Funkcja straty (ang. *loss function*) – funkcja kosztu, funkcja błędu.
- Przykład uczący (ang. *training example*) – rekord, próbka, rząd tabeli zestawu danych [9].

1.5. Splotowe(konwolucyjne) sieci neuronowe

Konwolucyjne sieci neuronowe (ang. *Convolutional neural networks - CNN*), zwane również splotowymi są najczęściej wykorzystywane w zadaniach związanych z klasyfikacją obrazów. Jest to rodzina modeli, których działanie inspirowane było korą wzrokową ludzkiego mózgu. Kluczowym zadaniem jest skuteczne wydobycie istotnych cech obrazu. Wielowarstwowe głębokie splotowe sieci neuronowe tworzą hierarchię cech, dzięki której uzyskiwane są wysokopoziomowe cechy szczegółowe [9].

1.6. Uczenie transferowe

Uczenie transferowe (ang. *transfer learning*) lub inaczej transfer wiedzy to jedna z technik uczenia maszynowego, dzięki której możliwe jest przenoszenie wiedzy uzyskanej w pokrewnym zadaniu i wykorzystanie jej do rozwiązania innego problemu. Eliminuje to potrzebę tworzenia modelu zupełnie od zera. Najczęściej wykorzystywane są pre-trenowane modele wielowarstwowych sieci neuronowych. Technika ta pozwala oszczędzić czas oraz ilość danych związanych z treningiem modelu [10].



Rysunek 1.3 Uczenie transferowe. [11]

2. Wykorzystane narzędzia i technologie

W celu skutecznej implementacji systemu konieczny był wybór oraz weryfikacja użyteczności odpowiednich narzędzi i technologii.

2.1. Język programowania i środowiska programistyczne

2.1.1. Język Python 3

Wykorzystano wszechstronny, interpretowany język programowania wysokiego poziomu, jakim jest *Python*. Charakteryzuje się czytelną i przejrzystą składnią, co sprawia, że jest często wybierany przez początkujących programistów. Jedną z kluczowych cech Pythona jest jego wsparcie dla wielu paradygmatów programowania, takich jak programowanie obiektowe, funkcjonalne oraz imperatywne. Python posiada bogaty ekosystem bibliotek i modułów, które ułatwiają realizację różnorodnych projektów, od analizy danych, przez tworzenie aplikacji webowych, po uczenie maszynowe. Dzięki swojej prostocie i wszechstronności, Python znajduje zastosowanie zarówno w projektach akademickich, jak i komercyjnych [12].



Rysunek 2.1 Logo języka Python. [13]

2.1.2. Platforma Google Colab

Platforma *Colaboratory (Colab)* to hostowana w chmurze usługa *Jupyter Notebook*, która nie wymaga konfiguracji do użycia, zapewnia bezpłatny dostęp do zasobów obliczeniowych (w tym *GPU* oraz *TPU*) i pozwala pisać oraz wykonywać kod w języku *Python* bezpośrednio w przeglądarce. Dzięki użyciu platformy *Colab* do procesu treningu modelu, czas wykonywania kodu zmieścił się w akceptowalnych ilościach [14].



Rysunek 2.2 Logo platformy Google Colab. [15]

2.1.3. JetBrains Pycharm Professional IDE

Zintegrowane środowisko programistyczne (*IDE*) *PyCharm* firmy *JetBrains* jest specjalnie dostosowane dla języka programowania *Python*. Wersja *Professional* dostarcza wiele przydatnych funkcjonalności. Umożliwia ono integrację z systemem kontroli wersji (*Git*), graficzny debugger, analizę i refaktoryzując kodu źródłowego, wbudowany terminal oraz wiele przydatnych dodatków [16]. Jego zastosowanie podczas tworzenia kodu aplikacji znacznie ułatwiło i przyspieszyło ten proces.



Rysunek 2.3 Logo środowiska Pycharm. [16]

2.1.4. Notatnik Jupyter

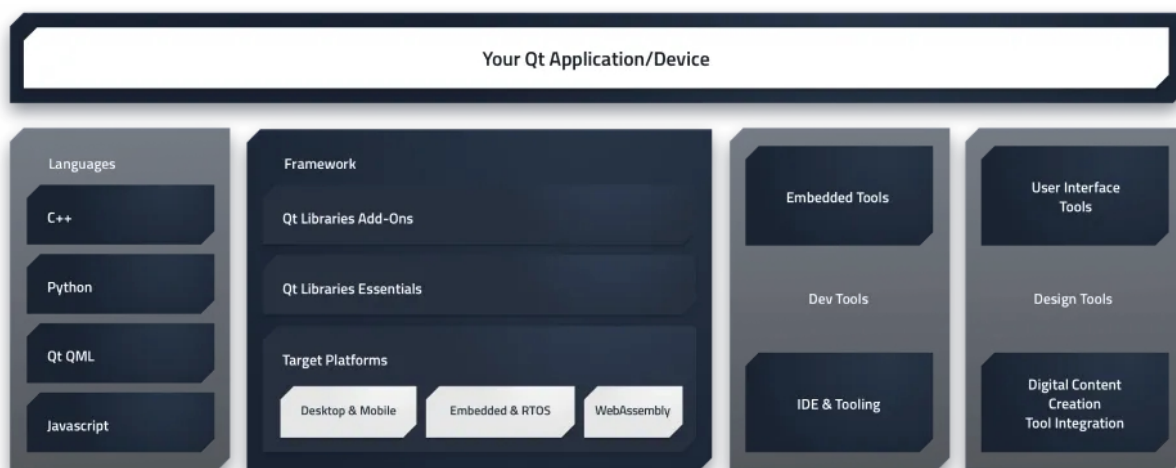
Notatnik Jupyter (ang. *Jupyter Notebook*) to aplikacja webowa, która umożliwia tworzenie zintegrowanej dokumentacji, kodu *Python*, obliczeń czy wizualizacji. Jest on szczególnie przydatny do analizy danych oraz zadań związanych z uczeniem maszynowym [17]. Wykorzystany on został w platformie *Colaboratory* podczas treningu modelu, co pozwoliło na podzielenie kodu na sekcje oraz integrację dokumentacji całego procesu treningu.



Rysunek 2.4 Logo Project Jupyter. [17]

2.1.5. Qt Framework

Użyto kompleksowy, intuicyjny i wieloplatformowy zestaw bibliotek do tworzenia aplikacji z interfejsem graficznym użytkownika (*GUI*), jakim jest *Qt Framework*. Jego uproszczony schemat blokowy widoczny jest na rysunku 2.5 Napisany jest w języku *C++*, ale oferuje wiązania dla innych języków (w tym *Python*). Posiada bogaty zestaw komponentów oraz dzięki wykorzystaniu *Meta-Object Compiler* oferuje *Signals&Slots*, czyli innowacyjny mechanizm komunikacji między obiektami. Wspiera on również wydajne renderowanie grafiki, obsługę dźwięku i obrazu, a także integrację z *OpenGL* [19].

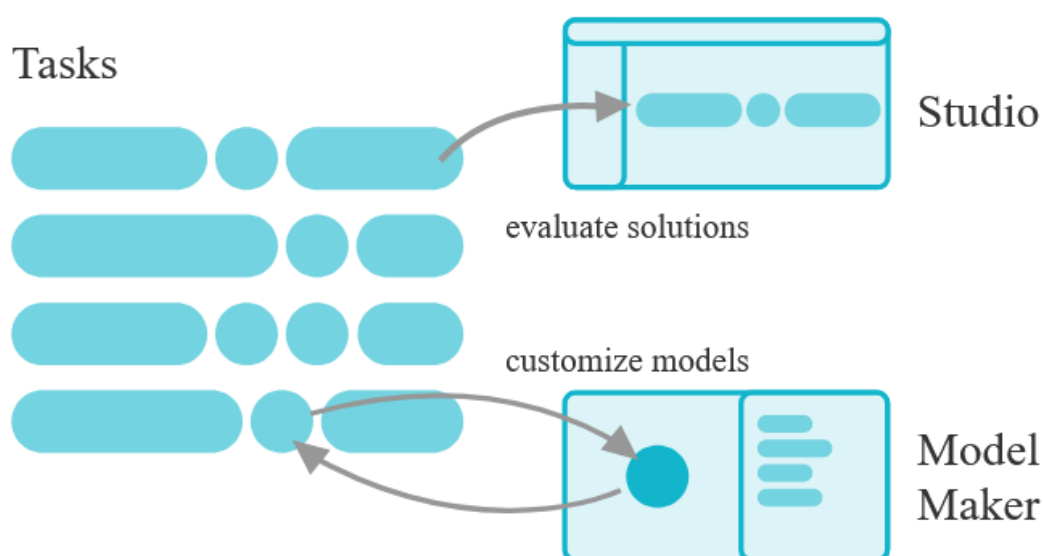


Rysunek 2.5 Schemat blokowy Qt Framework. [18]

2.2. Technologie sztucznej inteligencji i cyfrowego przetwarzania obrazu

2.2.1. Pakiet MediaPipe Solutions

Rozwiązanie *MediaPipe Solutions* to pakiet bibliotek i narzędzi dostarczany przez firmę *Google*. Pozwala on na szybką implementację technik sztucznej inteligencji oraz uczenia maszynowego w aplikacjach na wielu platformach [20].



Rysunek 2.6 Diagram części składowych pakietu *MediaPipe Solutions*. [20]

Części składowe pakietu *MediaPipe Solutions* (widoczne na rysunku 2.6):

- **MediaPipe Tasks:** interfejsy *API* i biblioteki na wielu platformach do wdrażania rozwiązań.
- **MediaPipe Models:** wytrenowane, gotowe do uruchomienia modele do użycia z każdym rozwiązaniem.
- **MediaPipe Model Maker:** dostosowywanie rozwiązań z użyciem własnych danych.
- **MediaPipe Studio:** wizualizacja oraz testy rozwiązania w przeglądarce. [20]

2.2.2. Biblioteka OpenCV

Biblioteka *OpenCV* jest dobrze zoptymalizowana oraz wieloplatformowa, służy do cyfrowego przetwarzania obrazu w czasie rzeczywistym o otwartym kodzie źródłowym. Bibliotekę zaprojektowano z myślą o efektywnym wykonywaniu obliczeń, zapewnia ona możliwość wykorzystania procesorów wielordzeniowych. Zawiera w sobie również podbibliotekę do zastosowań uczenia maszynowego. Stworzona została przez *Gary Bradski* pracującego w firmie *Intel Corporation* [8].



Rysunek 2.7 Logo biblioteki OpenCV. [21]

2.2.3. XNNPACK

Wysoce zoptymalizowana biblioteka operacji tensorowych, *XNNPACK*, została zaprojektowana do efektywnego wykonywania operacji tensorowych na procesorach *CPU*. Jest wykorzystywana przez *TensorFlow Lite* oraz *MediaPipe*, gdzie przyspiesza inferencję modeli uczenia maszynowego i zmniejsza zapotrzebowanie na zasoby sprzętowe [20]. Wspiera ona następujące architektury procesorów:

- ARM64 w *Android*, *iOS*, *macOS*, *Linux* oraz *Windows*
- ARMv7 (z *NEON*) w *Android*
- ARMv6 (z *VFPv2*) w *Linux*
- x86 oraz x86-64(do *AVX512*) w *Windows*, *Linux*, *macOS*, *Android*, oraz *iOS simulator*
- WebAssembly MVP
- WebAssembly SIMD
- RISC-V (*RV32GC* oraz *RV64GC*) [20]

2.2.4. Biblioteka TensorFlow

Biblioteka *TensorFlow* jest skalowalnym, wieloplatformowym oraz darmowym interfejsem programowania służącym do implementacji algorytmów uczenia maszynowego. Stworzona została przez firmę *Google* z myślą o zastosowaniach wewnętrznych, jednak udostępniono ją publicznie z otwartą licencją. Środowisko zawiera interfejsy, które mogą być obsługiwane przez różne języki programowania, w tym *Python*. Modele w wersji *Lite* są dostosowane do zastosowań mobilnych gdzie nie są dostępne duże zasoby obliczeniowe [9].



Rysunek 2.8 Logo biblioteki TensorFlow. [22]

2.2.5. Platforma Kaggle

Platforma konkursowa *Kaggle* służy do wyszukiwania, eksplorowania oraz udostępniania wysokiej jakości zbiorów danych i projektów uczenia maszynowego w ramach *Google*. Jest ona obecnie największą platformą crowdsourcingową przeznaczoną dla analityków danych oraz programistów uczenia maszynowego. Korzystanie z niej jest bezpłatne [24]. Umożliwia ona bezpośrednie pobranie zbioru danych w *Colab* używając pakietu (*Python*) *kaggle*. Jest ona źródłem pochodzenia zbioru danych (zdjęć gestów dłoni) użytych w procesie treningu [23].



Rysunek 2.9 Logo platformy Kaggle. [23]

2.3. System kontroli wersji i zarządzania projektem

2.3.1. System Git

System rozproszonej kontroli wersji, jakim jest *Git* utworzony został przez twórcę jądra *Linux* Linus Torvalds. Jest on prosty w nauce oraz zastosowaniu. Dzięki swoim funkcjonalnościom jest dużo lepszym wyborem niż inne narzędzia typu *SCM* (*Software Configuration Management*) takie jak *Subversion* [25]. Wykorzystany został do zarządzania oraz śledzenia zmian kodu źródłowego wraz z innymi plikami projektu.

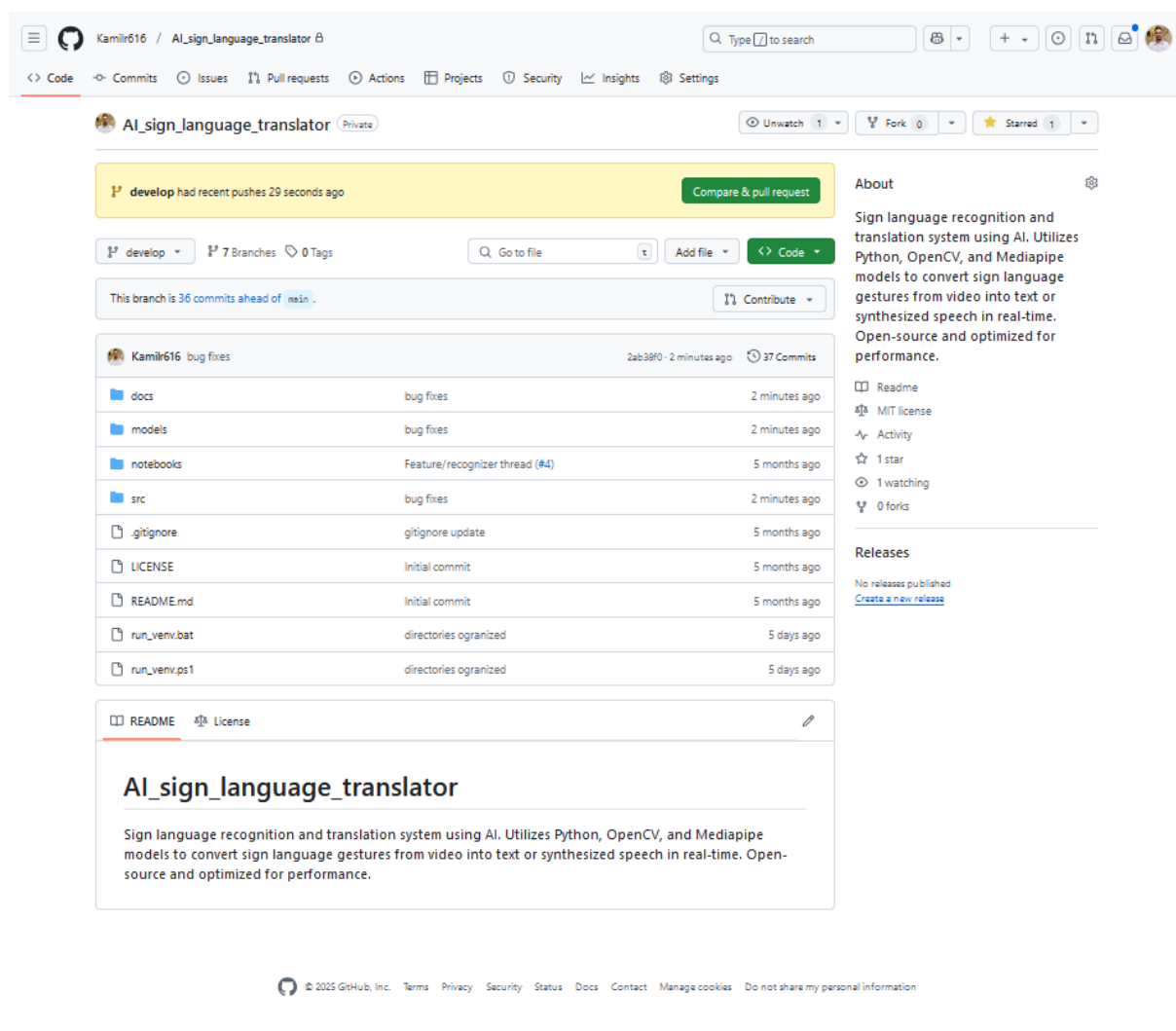


Rysunek 2.10 Logo systemu Git. [25]

2.3.2. Platforma GitHub

Platforma hostingowa *GitHub* firmy *Microsoft* implementuje system kontroli wersji *Git*, działając w „chmurze” [26]. Platforma wykorzystana została do przechowywania i zarządzania plikami projektu, co jest widoczne na rysunku 2.11. Przechowywanie kodu w repozytorium na platformie *GitHub* umożliwia:

- Prezentowanie lub udostępnianie swojej pracy
- Śledzenie i zarządzanie zmianami w kodzie w czasie
- Umożliwienie innym przeglądania kodu i zgłaszanie sugestii dotyczących jego ulepszenia
- Współpracę nad współdzielonym projektem bez obaw o konflikty podczas integracji kodu [26]



Rysunek 2.11 Zrzut ekranu repozytorium projektu na platformie GitHub.

2.4. Interfejs użytkownika i środowisko wizualne

2.4.1. Qt for Python

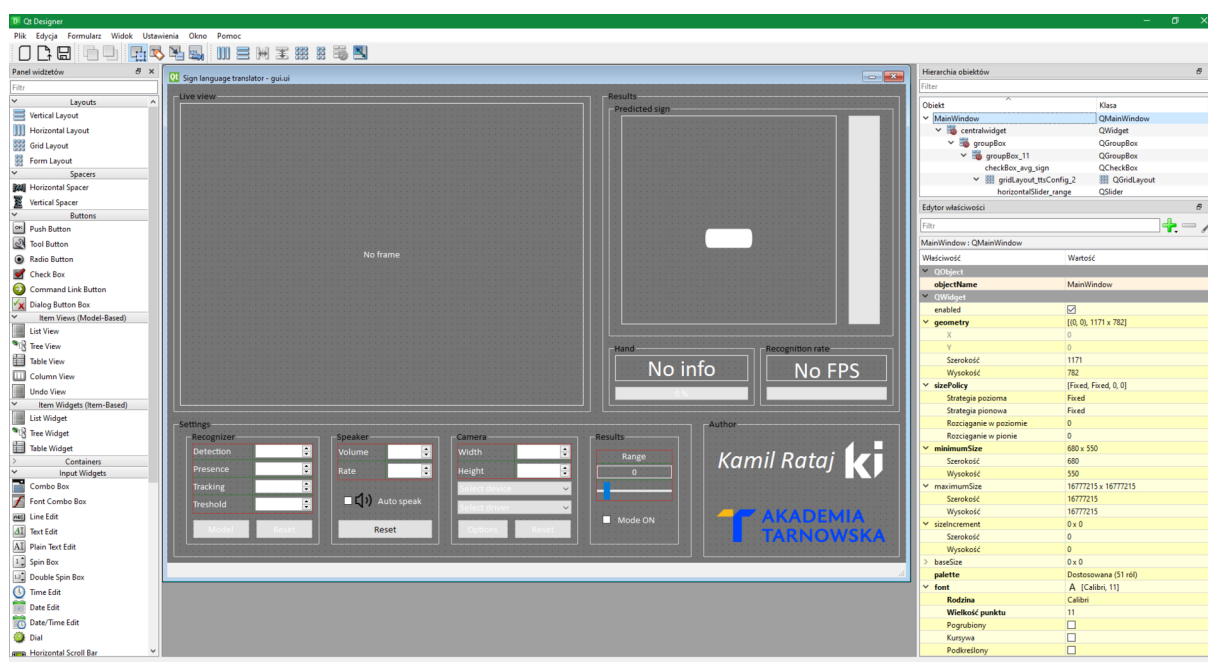
W celu wykorzystania biblioteki *Qt Framework* konieczna była implementacja wiązania językowego, jakim jest *Qt for Python*. Biblioteka *PySide6* jest najnowszą wersją tych wiązań i została oparta na *Qt 6*. Zapewnia ona pełny dostęp do *Qt API*, integrację z innymi bibliotekami *Python* oraz jest bardzo intuicyjna [19]. Aby korzystać z klas *Qt* w *Python* wymagany jest również generator wiązań *Shiboken6*, który automatycznie tworzy interfejs między kodem *C++ Qt*, a kodem *Python* [27]. Na rysunku 2.12 widoczny jest schemat blokowy architektury środowiska wykonawczego *Qt for Python*.



Rysunek 2.12 Schemat blokowy architektury środowiska *Qt for Python*. [19]

2.4.2. Qt Designer

Do projektu interfejsu graficznego użytkownika (GUI) wykorzystano narzędzie *Qt Designer*, widoczne na rysunku 2.13. Znacznie przyspiesza ono proces tworzenia oraz projektowania. Oferuje przydatną funkcję „przeciągnij i upuść” oraz automatycznie generuje pliki w języku znaczników XML [19]. Pliki te są konwertowane do kodu *Python* za pomocą narzędzia *pyside6-uic* [27].



Rysunek 2.13 Zrzut ekranu narzędzia Qt Designer.

2.4.3. QDarkStyle

Wykorzystano dedykowany styl graficzny *QDarkStyle* dla aplikacji napisanych z wykorzystaniem *Qt Framework*. Dostarcza on gotowe style dla wszystkich komponentów w ciemnej kolorystyce, nadaje im przez to nowoczesny oraz bardziej czytelny wygląd. Jest również bardzo prosty w implementacji [28].

2.5. Moduł syntezy mowy

Moduł syntezy mowy odpowiada za konwersję rozpoznanych gestów języka migowego na mowę syntetyzowaną, co umożliwia lepszą komunikację osób niesłyszących i słabosłyszących z otoczeniem.

2.5.1. Biblioteka pyttsx3

Biblioteka *pyttsx3* umożliwia generowanie mowy syntetyzowanej w trybie offline, bez konieczności korzystania z zewnętrznych usług internetowych. Zapewnia ona dostęp do systemowych silników mowy. Jest to istotne z punktu widzenia wydajności oraz niezależności systemu od połączenia z siecią. Pozwala ona na dostosowanie parametrów takich jak tempo czy głośność wypowiedzi [29].

2.5.2. Silnik(engine) syntezy mowy

Aplikacja wykorzystuje dostępny w systemie operacyjnym silnik syntezy mowy. W przypadku systemu *Windows* jest to *Microsoft Speech API (SAPI)*, który umożliwia korzystanie z wbudowanych głosów, takich jak *Microsoft Zira*.



Rysunek 2.14 Logo biblioteki pyttsx3. [29]

3. Trening modelu klasyfikacyjnego

W celu spełnienia założeń pracy konieczny był trening modelu MediaPipe z zastosowaniem techniki uczenia transferowego oraz odpowiednich narzędzi.

3.1. Środowisko

Trening modelu został przeprowadzony wykorzystując platformę *Google Colaboratory* wraz z usługą *Jupyter Notebook*. Zastosowanie środowiska o właściwościach widocznych w tabeli 3.1. pozwoliło to na uzyskanie zadowalającej wydajności obliczeniowej oraz czasu wykonywania programu, który mieścił się w darmowych limitach zużycia.

Tabela 3.1. Właściwości środowiska Colaboratory.

Typ środowiska	Akcelerator sprzętowy	Pamięć RAM	Dysk twardy
<i>Python 3</i>	<i>Intel Xeon CPU 2,20 [GHz]</i>	12,7 [GB]	107,7 [GB]

3.2. Zbiór danych treningowych

Zbiór zawiera około 87 tys. zdjęć dłoni w rozdzielczości 200x200 [px] podzielonych na 29 klas (znajdujących się w osobnych podfolderach). Klasy kolejno reprezentują znaki amerykańskiego alfabetu palcowego (A-Z), znak odstępu, znak kasowania oraz brak znaku [23]. Przykładowe zdjęcia ze zbioru widoczne są na rysunku 3.1.



Rysunek 3.1 Przykładowe obrazy ze zbioru treningowego. [23]

3.2.1. Źródło

Dane użyte do treningu pochodzą z platformy *Kaggle*. Zbiór został przygotowany przez użytkownika *grassknotted* (*Akash Nagaraj*) [23]. Token dostępu do *API* został pobrany z profilu użytkownika *Kaggle* w formie pliku *kaggle.json*, a jego przykładowa zawartość jest widoczna na listingu 3.1.

Listing 3.1. Przykładowa zawartość pliku kaggle.json.

```
{"username": "kamilrataj", "key": "fa56aff650af17fcff4a0c1c9cfff510"}
```

3.2.2. ETL zbioru danych

Wszystkie obrazy wykorzystane w procesie uczenia zostały bezpośrednio pobrane za pomocą pakietu (*Python*) *kaggle*, który zapewnia dostęp do *API* platformy. Konieczny był do tego celu token dostępu *kaggle.json* (listing 3.1.). Obrazy po pobraniu są przechowywane tymczasowo w przestrzeni dyskowej platformy *Colaboratory*. Konieczne jest usunięcie folderów zawierających klasy które nie będą używane w procesie treningu, czyli znak odstępu, znak kasowania oraz niestandardowe znaki „J” oraz „Z”. Dostosowano również nazwę folderu *nothing* zawierającego zdjęcia z brakiem znaku. Kod odpowiedzialny za to zadanie widoczny jest na listingu 3.2.

Listing 3.2. Pobranie oraz zapis obrazów używanych w treningu.

```
!mkdir -p ~/.kaggle
!cp kaggle.json ~/.kaggle/
!chmod 600 ~/.kaggle/kaggle.json

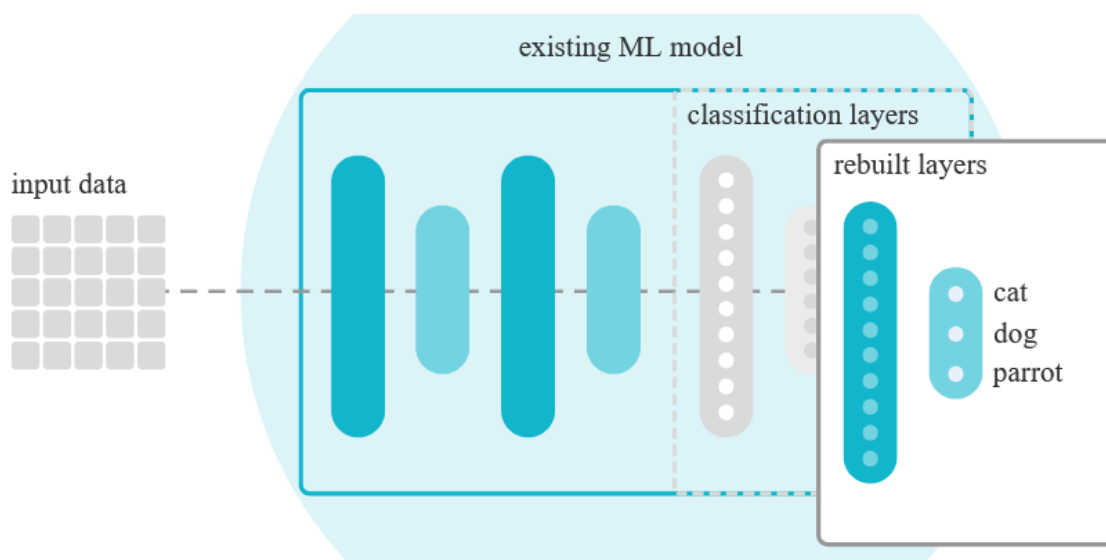
import kaggle
import shutil

kaggle_dataset_name = "grassknotted/asl-alphabet"

try:
    kaggle.api.dataset_download_files(kaggle_dataset_name, path='datasets/',
    unzip=True)
    dataset_train_path = "datasets/asl_alphabet_train/asl_alphabet_train"
    os.rename(f"{dataset_train_path}/nothing", f"{dataset_train_path}/none")
    shutil.rmtree(f"{dataset_train_path}/del", ignore_errors=True)
    shutil.rmtree(f"{dataset_train_path}/space", ignore_errors=True)
    shutil.rmtree(f"{dataset_train_path}/J", ignore_errors=True)
    shutil.rmtree(f"{dataset_train_path}/Z", ignore_errors=True)
    print(f"Successfully processed {kaggle_dataset_name} files!")
except Exception as e:
    print(f"Failed to load {kaggle_dataset_name}: {e}!")
```

3.3. MediaPipe Model Maker

Kreator modeli MediaPipe jest narzędziem pozwalającym na dostosowanie istniejących modeli uczenia maszynowego. Stanowi on szybszą oraz prostszą alternatywę dla tworzenia nowego modelu. Pozwala on na uzyskanie zadowalających rezultatów używając mniejszej ilości danych treningowych, dzięki znacznemu wykorzystaniu dotychczasowej logiki modelu. Wykorzystuje on (w tym przypadku) modele *palm_detection* oraz *hand_landmark* do uzyskania tensorów określających punkty orientacyjne dłoni z obrazów. W tym etapie uzyskana została struktura danych *Dataset* (listing 3.3.), która nie zawiera obrazów, a już osadzone gesty (ang. *embeddings*) [30].



Rysunek 3.2 Usuwanie ostatnich warstw modelu i odbudowanie z nowymi danymi. [30]

Listing 3.3. Utworzenie struktury *Dataset*.

```
data = gesture_recognizer.Dataset.from_folder(  
    dirname=dataset_train_path,  
    hparams=gesture_recognizer.HandDataPreprocessingParams(shuffle=True,  
min_detection_confidence=0.55)
```

3.4. Podział zbioru danych treningowych

Aby zapewnić skuteczność procesu uczenia transferowego oraz możliwość oceny wydajności modelu na nieznanych danych, korzystając z kodu widocznego na listingu 3.4. zbiór danych treningowych został podzielony na trzy kluczowe podzbiory:

- Zbiór danych treningowych (80%) – wykorzystywany do nauki modelu. Model uczy się na podstawie tych danych, dostosowując swoje parametry w celu minimalizacji funkcji błędu.
- Zbiór danych walidacyjnych (18%) – służy do oceny modelu podczas treningu. Na jego podstawie dostrajane są hiperparametry modelu, co pomaga uniknąć przeuczenia (ang. *overfitting*).
- Zbiór danych testowych (2%) – wykorzystywany wyłącznie po zakończeniu treningu w celu oceny ogólnej skuteczności modelu na nowych, nieznanych danych.

Listing 3.4. Podział zbioru danych treningowych.

```
train_data, rest_data = data.split(0.8)
validation_data, test_data = rest_data.split(0.9)
```

3.5. Parametryzacja

Kreator modeli *MediaPipe Model Maker* pozwala na dostosowanie wielu hiperpatametrów treningu mających kluczowy wpływ na dokładność i precyzję modelu.

3.5.1. Hiperparametry modelu

Dostępne są dwa hiperpatametry w klasie *ModelOptions* wpływające bezpośrednio na model:

- ***dropout_rate*** – parametr określający ułamek jednostek wejściowych odrzucanych podczas treningu. Służy on jako mechanizm regularyzacyjny w warstwie *dropout*, zapobiegając nadmiernemu dopasowaniu modelu.
- ***layer_widths*** – lista szerokości ukrytych warstw modelu. Każdy element odpowiada nowej ukrytej warstwie o zadanej liczbie neuronów. Między warstwami stosowane są mechanizmy normalizacji wsadowej(*BatchNorm*), odrzucania jednostek(*Dropout*) oraz funkcji aktywacji *ReLU*.

3.5.2. Hiperparametry treningu

Dostępne hiperparametry w klasie *HParams* wpływające na proces treningu:

- *learning_rate* – współczynnik uczenia stosowany w algorytmie spadku gradientu.
- *batch_size* – rozmiar partii danych używanej podczas jednej iteracji treningowej.
- *epochs* – liczba iteracji treningowych nad całym zbiorem danych.(Epoki)
- *steps_per_epoch* – opcjonalny parametr określający liczbę kroków treningowych na epokę.
- *shuffle* – flaga decydująca, czy dane wejściowe są losowo mieszane przed treningiem.
- *lr_decay* – parametr określający tempo zmniejszania współczynnika uczenia podczas treningu.
- *gamma* – parametr *gamma* wykorzystywany w funkcji straty *focal loss*.

3.5.3. Zastosowane hiperparametry

W ramach pracy przeprowadzono wiele prób treningu z różnymi hiperparametrami, osiągając różne rezultaty. Dla wymienionych w listingu 3.5. wartości, ewaluacja modelu po pomyślnym treningu dała najwyższe wyniki precyzji oraz dokładności.

Listing 3.5. Hiperparametry treningu.

```
_dropout_rate = 0.075
_layer_width = [128, 64, 32]
_learning_rate = 0.001
_batch_size = 16
_epochs = 70
_steps_per_epoch = None
_shuffle = True
_lr_decay = 0.95
_gamma = 2
```

3.6. Ewaluacja modelu

Po treningu przeprowadzono pomyślną ewaluację modelu, która jest opisana w podrozdziale 5.1.

3.7. Eksport modelu

Model został przekonwertowany do formatu *TFLite* oraz wraz z etykietami klas wyeksportowany do pliku *gesture_recognizer_asl.task*. Po pobraniu można z niego korzystać w rozwiązaniu *MediaPipe Custom Gesture Recognizer*, a także po odpowiednim obsłużeniu w innych systemach, które nie oferują dużej mocy obliczeniowej (np. urządzenia mobilne). Model jest przechowywany w formie pliku z rozszerzeniem *.task* co pozwala na jego bezpośrednie użycie. Plik zajmuje stosunkowo mało przestrzeni dyskowej (około 8 [MB]). Możliwe jest również pobranie pliku z etykietami klas (ang. *labels*).

3.8. Podsumowanie

Uzyskany model składa się z warstw opisanych w tabeli 3.2. oraz cechuje się następującą liczbą parametrów:

- Łączna liczba parametrów: 29,081 (113,60 [KB])
- Liczba parametrów trenowalnych: 28,377 (110,85 [KB])
- Liczba parametrów nietrenowalnych: 704 (2,75 [KB])

Dzięki skonfigurowaniu hiperparamtru *layer_width* (listing 3.5.) sieć zawiera trzy warstwy typu *Dense*, które zmniejszają wymiarowość danych.

Tabela 3.2. Opis warstw modelu.

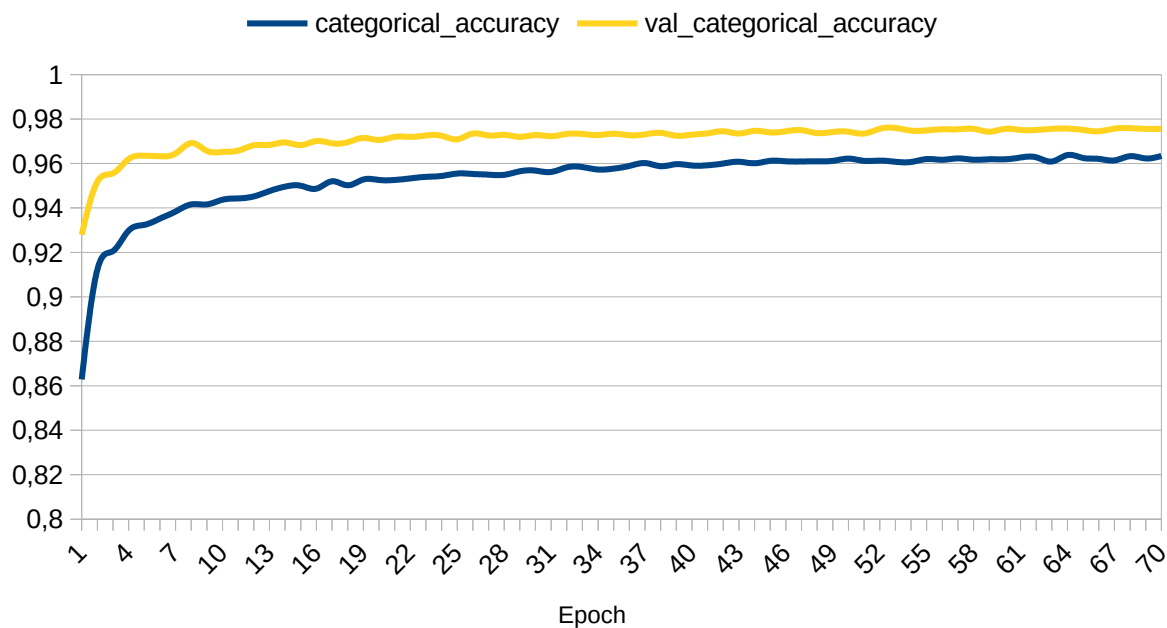
Warstwa	Typ	Kształt wyjściowy	Liczba parametrów
hand_embedding	InputLayer	(None, 128)	0
batch_normalization	BatchNormalization	(None, 128)	512
re_lu	ReLU	(None, 128)	0
dropout	Dropout	(None, 128)	0
custom_gesture_recognizer_0	Dense	(None, 128)	16,512
batch_normalization_1	BatchNormalization	(None, 128)	512
re_lu_1	ReLU	(None, 128)	0
dropout_1	Dropout	(None, 128)	0
custom_gesture_recognizer_1	Dense	(None, 64)	8,256
batch_normalization_2	BatchNormalization	(None, 64)	256
re_lu_2	ReLU	(None, 64)	0
dropout_2	Dropout	(None, 64)	0
custom_gesture_recognizer_2	Dense	(None, 32)	2,080
batch_normalization_3	BatchNormalization	(None, 32)	128
re_lu_3	ReLU	(None, 32)	0
dropout_3	Dropout	(None, 32)	0
custom_gesture_recognizer_out	Dense	(None, 25)	825

3.9. Wykres przebiegu procesu treningu

Przedstawione w tym podrozdziale wykresy stanowią cenne narzędzie diagnostyczne, które umożliwiają identyfikację potencjalnych problemów oraz optymalizację procesu uczenia modelu. Dzięki wizualizacji kluczowych metryk, takich jak funkcja straty, dokładność i współczynnik uczenia, możemy analizować dynamiczny charakter procesu uczenia w kontekście zmieniających się epok. Pozwala to na bieżąco monitorować postępy modelu, oceniać jego zdolność do generalizacji oraz w razie potrzeby modyfikować parametry treningu w celu uzyskania lepszych rezultatów [10]. Wykresy zostały utworzone za pomocą narzędzia *LibreOffice Calc*, na podstawie danych z procesu treningowego.

3.9.1. Wykres dokładności (ang. *accuracy*)

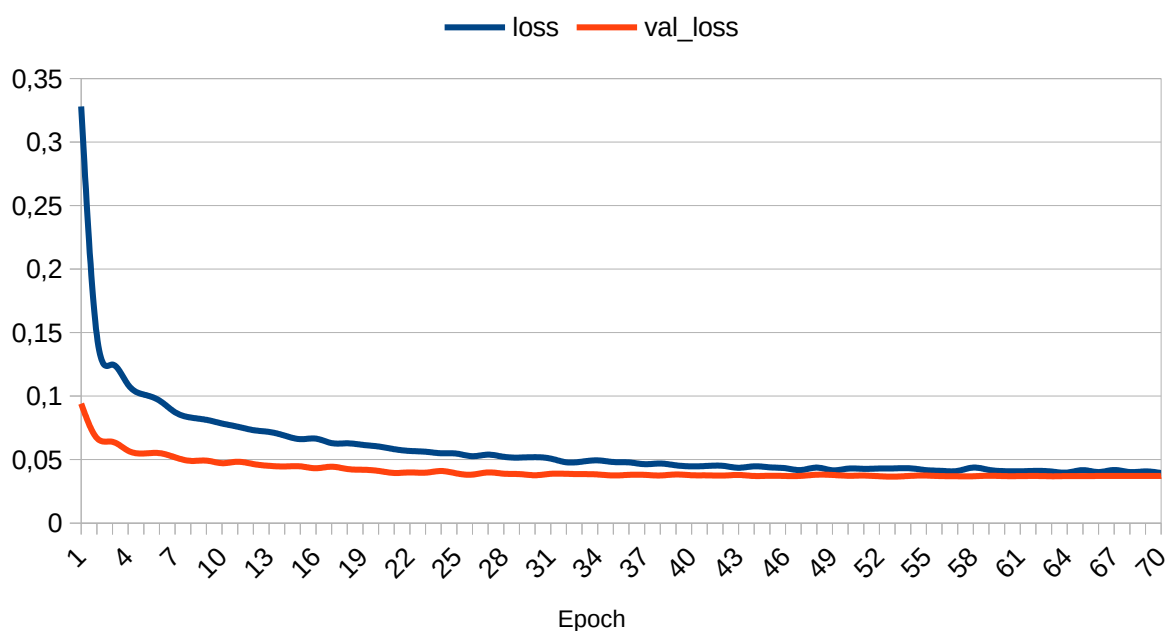
Wykres widoczny na rysunku 3.3 przedstawia, jak zmienia się dokładność modelu (dla zbioru treningowego i walidacyjnego) w zależności od epoki. Obserwacja tych wykresów pozwala ocenić, jak dobrze model radzi sobie z klasyfikacją danych i czy osiąga zadowalające wyniki [10].



Rysunek 3.3 Wykres dokładności modelu.

3.9.2. Wykres funkcji straty (ang. *loss*)

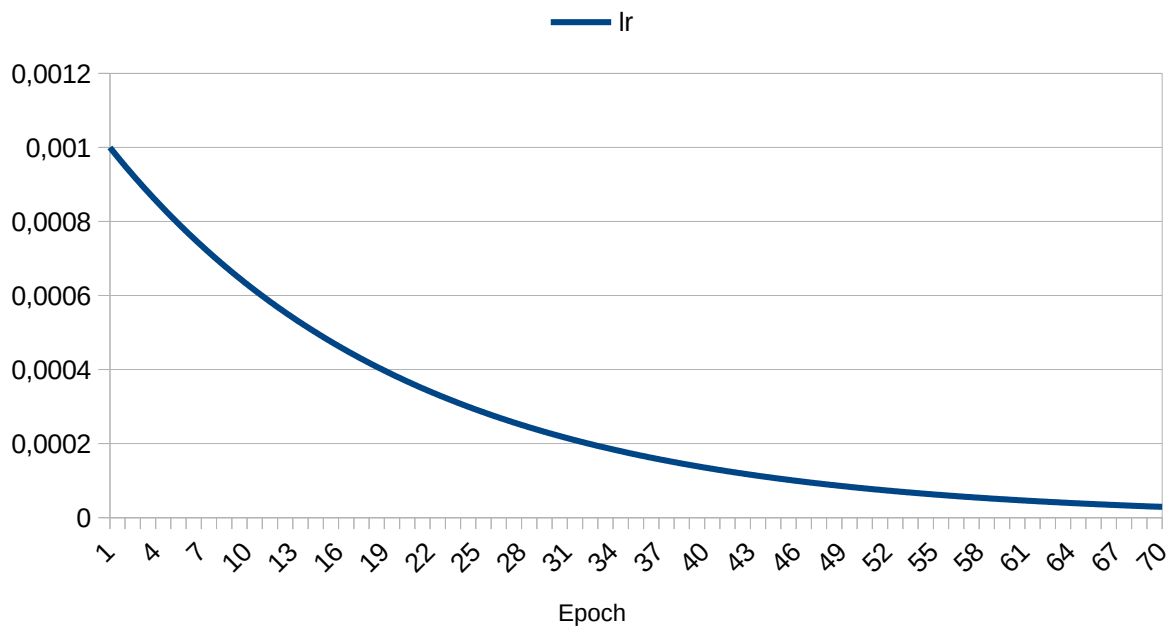
Wykres przedstawiony na rysunku 3.4 pokazuje jak zmienia się wartość funkcji straty (zarówno dla zbioru treningowego, jak i walidacyjnego) w zależności od epoki. Analiza tych wykresów pozwala ocenić, czy model uczy się poprawnie, czy też występują problemy takie jak przeuczenie (ang. *overfitting*) lub niedouczenie (ang. *underfitting*) [10].



Rysunek 3.4 Wykres funkcji straty.

3.9.3. Wykres współczynnika uczenia(ang. *learning rate*)

Wykres przedstawia, jak zmienia się wartość współczynnika uczenia w zależności od epoki. Analiza zmian widocznych na rysunku 3.5 pozwala zaobserwować, czy algorytm optymalizacji dynamicznie dostosowuje ten parametr oraz jego wartości co jest kluczowe dla efektywnego uczenia modelu [10].



Rysunek 3.5 Wykres współczynnika uczenia.

4. Implementacja systemu

Rozdział opisuje działanie systemu. Przedstawiono w nim architekturę oprogramowania oraz szczegóły dotyczące implementacji kluczowych modułów i wykorzystywanych technologii, takich jak przetwarzanie obrazu, rozpoznawanie gestów i translacja znaków języka migowego na tekst oraz mowę syntetyzowaną. Omówiono również integrację algorytmów sztucznej inteligencji w systemie.

4.1. Środowisko uruchomieniowe

Uruchamianie aplikacji jest możliwe w systemach operacyjnych *Windows* oraz po odpowiedniej adaptacji również *MacOS* i *Linux*.

4.1.1. Wirtualne środowisko Python

W celu zapewnienia izolacji zależności oraz łatwej konfiguracji środowiska uruchomieniowego wykorzystano wirtualne środowisko *Python* (*venv*). Pozwala ono na instalację wymaganych bibliotek bez ingerencji w globalne pakiety systemowe. Do implementacji systemu użyto języka *Python 3.10*. Widoczny na listingu 4.1. skrypt wsadowy (*.bat*) został stworzony w celu automatycznego uruchamiania aplikacji *Python* w odpowiednim środowisku wirtualnym. Jego zadaniem jest zmiana katalogu roboczego, aktywacja środowiska *venv* oraz uruchomienie głównego skryptu aplikacji. Dostępny jest również skrypt *PowerShell* (*.ps1*). Skrypty te są przydatne do uruchamiania aplikacji bez konieczności ręcznego aktywowania środowiska wirtualnego. Wystarczy dwukrotnie kliknąć plik *run_venv.bat* lub *run_venv.ps1*, aby rozpocząć działanie programu.

Listing 4.1. Skrypt startowy run_venv.bat do uruchamiania aplikacji.

```
@echo off

REM Change directory
cd .\src

REM Activate the virtual environment
call ..\venv\Scripts\activate.bat

REM Run the Python script
python .\main.py
```

4.1.2. Aplikacja wykonywalna (.exe)

W celu uproszczenia procesu dystrybucji aplikacji, szczególnie w przypadku użytkowników, którzy nie są zaznajomieni z konfiguracją środowisk wirtualnych (*venv*) czy instalacją zależności Pythona, zastosowano narzędzie *auto-py-to-exe*. Jest to interfejs graficzny dla popularnego narzędzia *PyInstaller*, które umożliwia tworzenie samodzielnych aplikacji wykonywalnych (.exe) z projektów *Python*, eliminując potrzebę posiadania zainstalowanego *Pythona* i zależności na maszynie docelowej. Użycie *auto-py-to-exe* do konwersji aplikacji w Pythonie do samodzielnego pliku wykonywalnego(.exe) uprościło proces dystrybucji aplikacji, eliminując potrzebę instalowania środowiska oraz zależności *Pythona* na systemach użytkowników. Aplikacja w tej formie zajmuje 375 [MB] przestrzeni dyskowej.

4.1.3. Urządzenie rejestrujące

Do działania aplikacji wymagana jest kamera *RGB* dostępna w interfejsie *USB* lub inne urządzenie rejestrujące dostępne w systemie operacyjnym. Zalecana rozdzielczość to 640x480 [px], a zalecana liczba rejestrowanych klatek obrazu na sekundę (*FPS*) to 30. Aplikacja wspiera również technologię *DirectShow*, która dobrze się sprawdza podczas przetwarzania obrazu w czasie rzeczywistym. Gdy urządzenie oferuje możliwość konfiguracji ustawień natywnych dostępne jest również okno z tymi ustawieniami. Możliwy jest wybór innej kamery spośród dostępnych w rozwijanej liście znajdującej się w sekcji *Settings/Camera*.

4.1.4. Biblioteka TensorFlow Lite

W ramach implementacji systemu wykorzystano *TensorFlow Lite* (*TFLite*) jako zoptymalizowaną wersję frameworka *TensorFlow*, przeznaczoną do uruchamiania modeli uczenia maszynowego na urządzeniach o ograniczonych zasobach obliczeniowych. Jest to kluczowe w kontekście środowiska uruchomieniowego, ponieważ aplikacja wymaga wydajnego przetwarzania obrazu i rozpoznawania gestów w czasie rzeczywistym.

4.2. Przechwytywanie obrazu

Klasa *CameraApp* jest odpowiedzialna za przechwytywanie klatek obrazu z urządzenia rejestrującego, takiego jak np. kamera internetowa *USB*. Wykorzystuje ona obiekt *VideoCapture* (listing 4.2.) z biblioteki *OpenCV*. Klasa obsługuje różne interfejsy kamery, w tym *DirectShow*. Pozwala również na dostosowanie parametrów urządzenia. Metoda *read()* widoczna w listingu 4.3. zwraca krotkę z aktualnym znacznikiem czasowym (ang. *timestamp*) oraz najnowszą klatkę obrazu (*numpy.ndarray*) lub w przypadku niepowodzenia *None*. Klatka obrazu jest konwertowana z przestrzeni barw *BGR* na *RGB*.

Listing 4.2. Konstruktor klasy *CameraApp*.

```
class CameraApp:
    def __init__(self, **kwargs):
        """
        Initializes the CameraApp instance.

        Args:
            **kwargs: Keyword arguments for configuring the camera.
                     Expected keys:
                     - fd (int or str): File descriptor or device index.
        """
        try:
            self.cap = cv2.VideoCapture()
            self.open(kwargs["fd"])
        except Exception:
            logging.error("Error while initializing the camera")
            self.destroy()
        finally:
            self.configure(**kwargs)
```

Listing 4.3. Metoda *read()* klasy *CameraApp*.

```
def read(self):
    """
    Captures a frame from the camera.

    Returns:
        tuple: A tuple containing:
            - int: The current timestamp in nanoseconds.
            - numpy.ndarray or None: The captured frame in RGB format,
              or None if the capture failed.
    """
    if self.cap.isOpened():
        ret, frame = self.cap.read()
        if ret:
            return time.time_ns(), cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
        else:
            logging.warning("Failed to capture frame.")
    else:
        logging.warning("Camera is not opened.")

    return time.time_ns(), None
```

4.3. Analiza obrazu

Rozwiązanie *MediaPipe Hands* odpowiedzialne jest za rozpoznawanie gestów dłoni w czasie rzeczywistym. Składa się z dwuetapowego pipeline'u, jako dane wejściowe przyjmuje obraz (*numpy.ndarray*) 2D w przestrzeni barw *RGB* [31]. Przechwycone klatki obrazu są przekazywane po kolei (z weryfikacją znacznika czasowego) i wywołują asynchroniczną metodę *recognize_async()* obiektu *recognizer*, która rozpoczyna proces rozpoznawania dłoni i klasyfikacji gestu w klasie *RecognizerApp*. Kod realizujący to zadanie widoczny jest na listingu 4.4.

Listing 4.4. Metoda *recognize_frame()* klasy *RecognizerApp*.

```
def recognize_frame(self):
    """
    Captures a frame from the camera and processes it for gesture
    recognition.

    Returns:
        None: The function does not return a value but processes the frame
        asynchronously.
    """
    if self.cap.is_closed() or self.recognizer is None:
        logging.warning("Camera is not opened or recognizer is not
initialized.")
        return

    timestamp, image = self.cap.read()
    while timestamp <= self.last_timestamp:
        logging.warning(f"Skipping outdated frame: {timestamp}")
        timestamp, image = self.cap.read()

    if image is not None:
        try:
            mp_image = Image(image_format=ImageFormat.SRGB,
data=image.astype(np.uint8))
            self.recognizer.recognize_async(mp_image, timestamp //
1_000_000)
        except Exception as e:
            logging.error(f"Exception in recognizer: {e}")
        finally:
            self.last_timestamp = timestamp
    else:
        logging.warning("No valid image to recognize.")
```

4.3.1. Wykrywanie dłoni

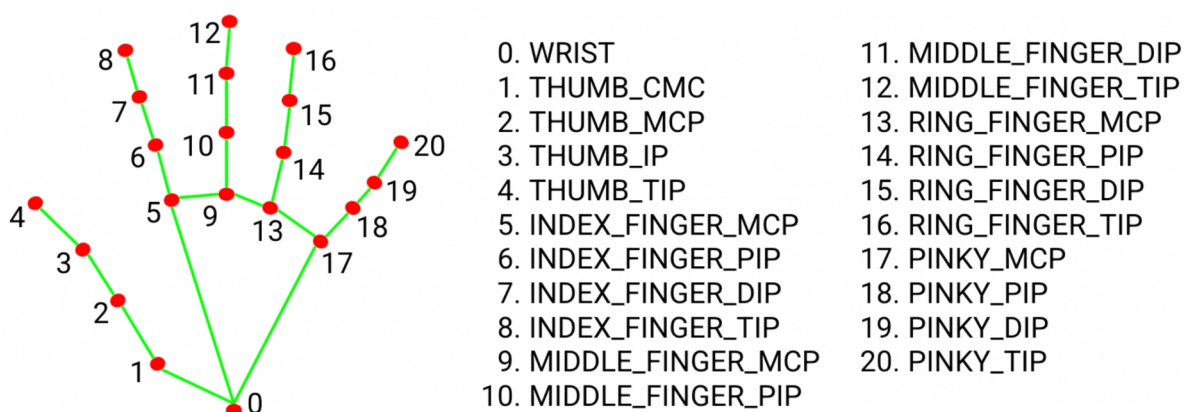
Za wykrywanie dłoni w obrazie odpowiedzialny jest *Hand detector model*, ustala on orientację dłoni oraz granice regionu, w którym znajduje się dłoń. Dzięki temu nie ma konieczności analizy całego obrazu w kolejnym kroku. Używana konwolucyjna sieć neuronowa stosuje architekturę *Single-shot detector*, jest to zaawansowany algorytm pozwalający na identyfikację i lokalizację obiektów na obrazie w jednym przejściu co znacznie przyspiesza ten proces oraz czyni go wydajniejszy obliczeniowo. Dzięki temu nadaje się on do używania w czasie rzeczywistym. Podczas analizy obrazu obsługuje on dużą zmienność skali dłoni oraz przypadki częściowego zakrycia dłoni. Model zwraca istotną informację o lokalizacji dłoni w obrazie. Model ten jest używany(poza pierwszą klatką obrazu) w przypadku niepowodzenia śledzenia [32]. W kolejnym kroku z wykorzystaniem algorytmu *NMS* tworzony jest wycinek obrazu z samą dłonią, który jest przetwarzany w następnym modelu [31].

4.3.2. Ustalanie punktów orientacyjnych i śledzenie

Po pomyślnym wykryciu lub na podstawie lokalizacji dłoni z poprzedniej klatki przycięty obraz trafia do modelu *Hand tracking model*, który przeprowadza precyzyjną lokalizację 21 kluczowych punktów orientacyjnych (2D oraz 3D) na dłoni. Punkty te odpowiadają stawom palców oraz śródręcza. Uzyskane punkty orientacyjne dłoni reprezentuje tensor o wymiarze 1x63 w dwóch wersjach. Opis tych punktów widoczny jest na rysunku 4.1 Określa on też ręczność dłoni jako skalar (*float*).

Model jest konwolucyjną siecią neuronową o regresyjnej architekturze [31]. Najważniejsze jego cechy to:

- zmniejszona potrzeba augmentacji danych oraz skupienie sieci na precyzję przewidywania współrzędnych,
- bezpośrednia predykcja współrzędnych z wykorzystaniem regresji,
- odporność na częściowo zakryte dłonie,
- minimalizacja obciążenia obliczeniowego programu, dzięki śledzeniu dłoni.



Rysunek 4.1 Opis punktów orientacyjnych dłoni. [32]

4.4. Klasyfikacja gestów

Następnie informacje trafiają do dwuetapowego rozwiązania *MediaPipe Hand Gesture Classification*, które zostało odpowiednio dostosowane, wykorzystując kreator modeli *MediaPipe Model Maker* oraz technikę uczenia transferowego. W efekcie uzyskiwany jest wektor zawierający wartości prawdopodobieństwa dla każdego rozpoznawalnego gestu (daktylogramu) oraz prawdopodobieństwo braku gestu. Modele nie przetwarzają bezpośrednio obrazu, a wykorzystują dane uzyskane w poprzednich krokach (*MediaPipe Hands*), co czyni rozwiązanie bardziej optymalnym oraz wydajniejszym obliczeniowo [33].

4.4.1. Model osadzeń (ang. embedding)

Model posiada architekturę regresyjną, jest siecią neuronową z blokami resztkowymi oraz połączeniami typu „każdy z każdym”. Dane wejściowe, które trafiają do modelu to dwa tensory zawierające informacje o punktach orientacyjnych dłoni oraz skalar reprezentujący ręczność, zostały one uzyskane w poprzednim modelu. Uzyskiwany w tym kroku jest tensor (128x1) zbudowany z przewidzianych osadzeń punktów orientacyjnych dłoni [33].

4.4.2. Model klasyfikacyjny

Model o architekturze klasyfikacyjnej opiera się o sieć neuronową z połączeniami typu „każdy z każdym”. Jest to ostatni etap przetwarzania informacji z wykorzystaniem technik sztucznej inteligencji. Danymi wejściowymi jest tensor osadzeń uzyskany w poprzednim modelu. Po zakończeniu cyklu zwracany jest wektor prawdopodobieństw. Wybierany jest znak dla którego prawdopodobieństwo jest największe [33].

4.5. Przygotowanie wyników

Po zakończeniu procesu klasyfikacji gestu, wywoływana jest asynchronicznie metoda `handle_result()` widoczna w listingu 4.5. Obliczana jest również ilość przetwarzanych klatek w ciągu 1 sekundy (*FPS*).

Listing 4.5. Metoda `handle_result` klasy `RecognizerApp`.

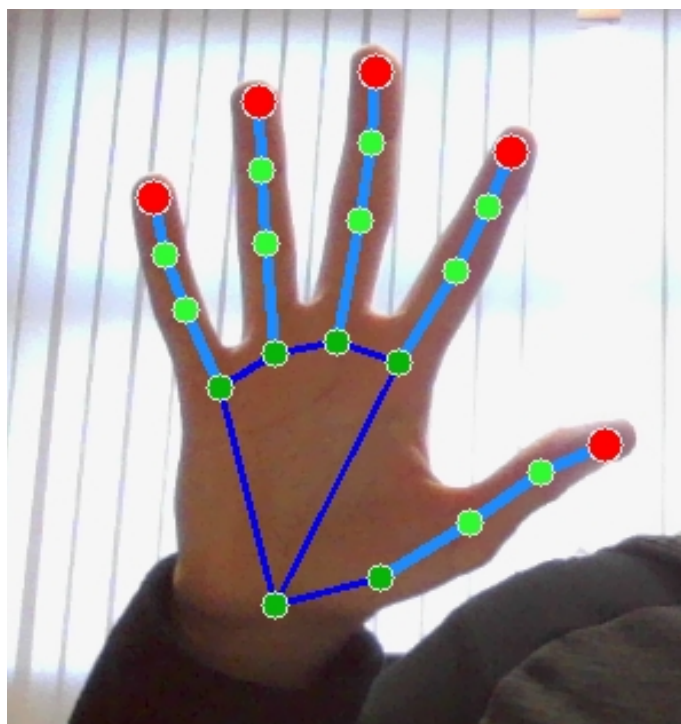
```
def handle_result(self, result: vision.GestureRecognizerResult,
output_image: Image, timestamp_ms: int):
    """
    Callback to process and emit the gesture recognition result.

    Args:
        result (GestureRecognizerResult): The recognition result containing
detected gestures.
        output_image (Image): The processed output image.
        timestamp_ms (int): The timestamp of the result in milliseconds.
    """
    try:
        frame, text, category_name = self.process_recognition_result(
            output_image.numpy_view().copy(), result
        )
        self.calculate_fps()
        self.result_ready_signal.emit(create_scaled_qpixmap(frame), text,
category_name, self.fps)

        if self.recognizer:
            self.recognize_frame()
    except Exception as e:
        logging.error(f"Error handling recognition result: {e}")
```

4.5.1. Wizualizacja punktów orientacyjnych dłoni

W przypadku pomyślnej klasyfikacji gestu, na obraz w określonych wcześniej lokalizacjach rysowane są elementy graficzne reprezentujące stawy palców i śródręcza oraz połączenia między nimi. Wygląd tych elementów jest ustalany w pliku *custom_landmarks.py* i został skonfigurowany w celu zapewnienia wysokiej intuicyjności oraz widoczności. Na rysunku 4.2 widoczne naniesione na obraz opisywane punkty orientacyjne dłoni.



Rysunek 4.2 Wizualizacja punktów orientacyjnych dłoni na obrazie wynikowym.

4.5.2. Przygotowanie i formatowanie wyniku

Następnie, wybierane są istotne informacje oraz odpowiednio formatowane w celu dalszej prezentacji. Po zakończonym cyklu rozpoznawania, jeśli jest to konieczne obraz wynikowy jest skalowany do rozdzielczości 640x480 [px] z zachowaniem formatu oraz konwertowany do typu obsługiwanego przez *UI Qt (QPixmap)*, co jest widoczne w listingu 4.6. Po pomyślnej operacji obraz wynikowy przekazywany jest wraz z niezbędnymi informacjami do klasy *MainApp* z wykorzystaniem mechanizmu *Signals & Slots*.

Listing 4.6. Funkcja *create_scaled_qpixmap()* konwertująca i skalująca obraz wynikowy.

```
def create_scaled_qpixmap(frame: np.ndarray) -> QPixmap:
    """
    Converts a NumPy frame to QPixmap format and scales it to 640x480 only
    if needed.

    Args:
        frame (numpy.ndarray): The frame to convert (RGB).

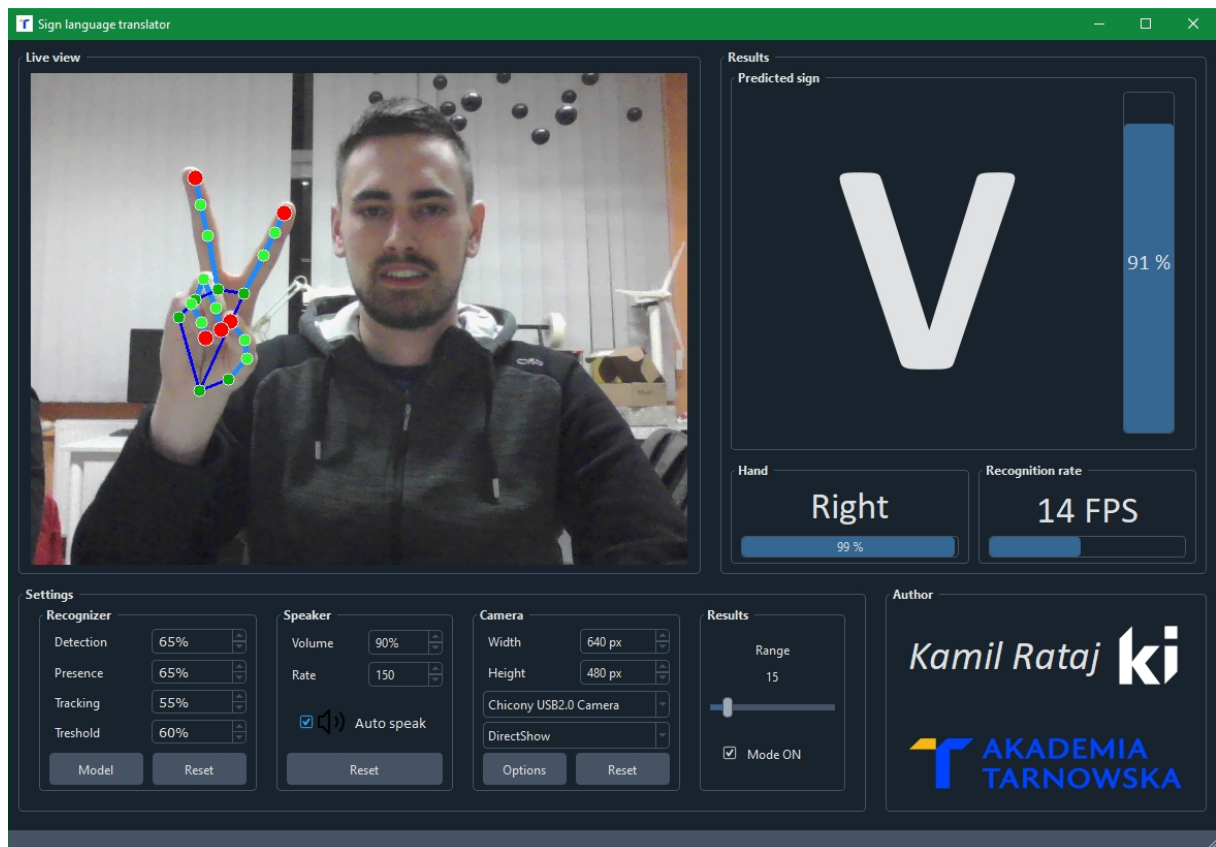
    Returns:
        QPixmap: The converted and possibly scaled QPixmap.
    """
    h, w, ch = frame.shape
    image = QImage(frame.data, w, h, ch * w, QImage.Format_RGB888)

    if (w, h) != (640, 480):
        scaled_image = image.scaled(QSize(640, 480), Qt.KeepAspectRatio,
Qt.FastTransformation)
    else:
        scaled_image = image

    return QPixmap.fromImage(scaled_image)
```

4.6. Interfejs użytkownika

Interfejs użytkownika został zrealizowany wykorzystując rozwiązanie *Qt for Python*. Interfejs graficzny aplikacji zawiera się w obrębie jednego okna systemowego, które prezentuje przetworzone dane w czasie rzeczywistym. Wygląd pełnego okna aplikacji widoczny jest na rysunku 4.3 System oferuje również konsolę wyświetlającą błędy oraz informacje diagnostyczne, co ułatwia korzystanie oraz monitorowanie działania aplikacji podczas jej użytkowania.



Rysunek 4.3 Zrzut ekranu interfejsu użytkownika.

4.6.1. Podgląd obrazu

Główna część interfejsu *Live view*, znajdująca się po lewej stronie wyświetla aktualny obraz na żywo z kamery, który jest wyskalowany do rozdzielczości 640x480 [px]. W przypadku pomyślnego rozpoznania dłoni również widoczna jest wizualizacja punktów orientacyjnych stawów palców i śródręcza.

4.6.2. Wyniki

Sekcja *Results* widoczna na rysunku 4.4 znajduje się po prawej stronie okna aplikacji, zawiera część główną *Predicted sign* z informacją dotyczącą aktualnie rozpoznanego znaku alfabetu wraz z poziomem pewności klasyfikacji znaku. Poniżej znajdują się informacje (*Hand*) dotyczące obecności i ręczności dłoni wraz z poziomem pewności detekcji dłoni oraz informacja o liczbie klatek (*Recognition rate*) przetworzonych przez model w ciągu 1 sekundy(*FPS*). Wartości liczbowe są wizualizowane za pomocą pasków postępu.



Rysunek 4.4 Panel wynikowy interfejsu użytkownika.

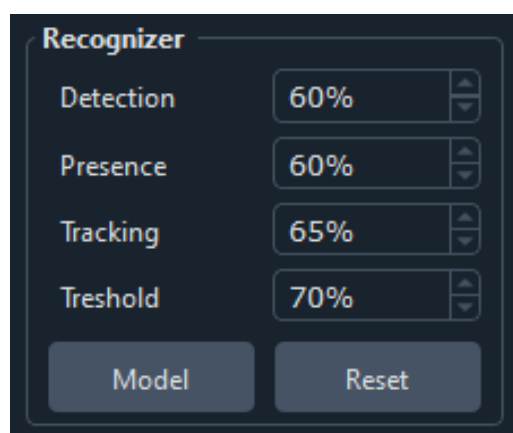
4.6.3. Ustawienia

Sekcja *Settings* składa się z 4 modułów, które można niezależnie konfigurować. Po zmianie parametrów w większości przypadków konieczne jest ponowne uruchomienie modułu za pomocą przycisków *Reset*.

4.6.3.1. Model

W tej części można dostosować wartości progowe [%] związane z modelem oraz rozpoznawaniem dłoni. Przycisk *Model* otwiera okno dialogowe pozwalające na wybór pliku z modelem (*.task*), po wybraniu prawidłowego modelu *reset* następuje automatycznie. Po wciśnięciu przycisku *Reset* przeprowadzana jest ponowna inicjalizacja klasy *GestureRecognizerApp* z nowymi parametrami. Konfigurowalne, w zakresie 0-100%, są następujące parametry:

- *Detection* - minimalna pewność wykrycia dłoni. Im wyższa wartość, tym mniejsze ryzyko fałszywych detekcji
- *Presence* - minimalna pewność obecności dłoni w obrazie. Wyższa wartość poprawia stabilność wykrywania
- *Tracking* - minimalna pewność śledzenia dłoni. Wyższa wartość zwiększa dokładność, ale zmniejsza elastyczność
- *Threshold* - minimalny próg pewności dla uznania gestu za poprawnie rozpoznany

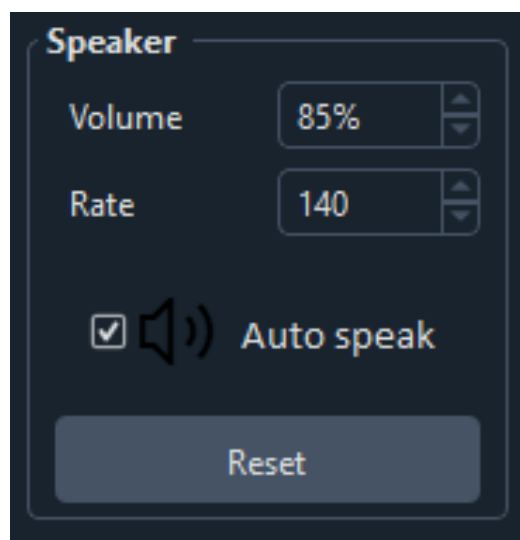


Rysunek 4.5 Panel ustawień modelu.

4.6.3.2. Synteza mowy

Aplikacja oferuje moduł konfiguracji syntezy mowy widoczny na rysunku 4.6. Zaznaczenie ikony głośnika z napisem *Auto speak* powoduje rozpoczęcie ciągłej syntezy mowy na podstawie rozpoznawanych znaków. Przycisk *Reset* powoduje zatrzymanie syntezy mowy oraz ponowną inicjalizację klasy *SpeakerApp* z nowymi ustawieniami. Dostępne są ustawienia syntezy mowy, które umożliwiają dostosowanie parametrów generowanego dźwięku do preferencji użytkownika. Obejmują one:

- *Volume* (głośność) – pozwala na regulację poziomu natężenia dźwięku, co umożliwia dostosowanie mowy do różnych warunków akustycznych.
- *Rate* (tempo) – określa szybkość wypowiadania słów, co pozwala na płynniejszą lub bardziej dynamiczną wymowę, w zależności od potrzeb użytkownika.



Rysunek 4.6 Panel ustawień syntezy mowy.

4.6.3.3. Urządzenie rejestrujące

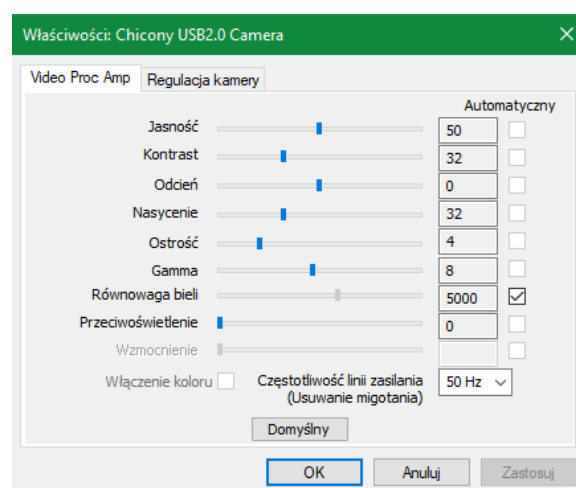
Na rysunku 4.8 widoczna jest sekcja ustawień urządzenia rejestrującego obraz. Dostępna jest możliwość wyboru urządzenia rejestrującego z listy. Lista zawiera tylko dostępne w systemie urządzenia rejestrujące i kamery. Konieczne jest również wybranie *API* kamery z listy dostępnych interfejsów. Lista zawiera również opcję automatycznego wyboru interfejsu - *Auto*. Podczas inicjalizacji domyślnym sterownikiem jest *DirectShow*. Ustawienia oferują również dostosowanie rozdzielczości kamery, wprowadzając odpowiednie wartości parametrów:

- *Width* (szerokość) w pikselach[px]
- *Height* (wysokość) w pikselach[px]

Przycisk *Options*, po kliknięciu powoduje otwarcie okna ustawień natywnych urządzenia/kamery widoczne na rysunku 4.7, jeśli jest ono dostępne. Przycisk *Reset* powoduje ponowne uruchomienie urządzenia/kamery z nowymi ustawieniami.



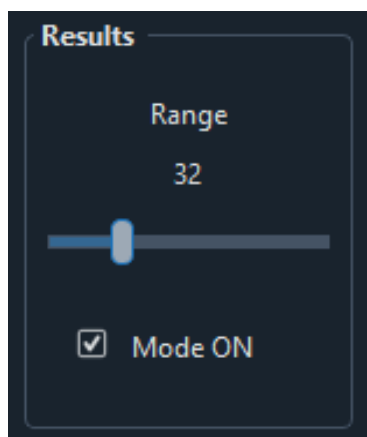
Rysunek 4.8 Panel ustawień urządzenia rejestrującego.



Rysunek 4.7 Okno ustawień natywnych kamery.

4.6.3.4. Wyniki

Sekcja *Results*, widoczna na rysunku 4.9 pozwala na zmianę trybu prezentowania wyników. W przypadku zaznaczenia *Mode ON* prezentowany wynik w sekcji *Predicted sign* to dominanta (ang. *mode*) ze zbioru tworzonego przez ostatnie wyniki rozpoznawania znaku, a wartość liczbową to średnia ważona prawdopodobieństwa dominującego znaku w zbiorze. Maksymalna wielkość tego zbioru, czyli parametr *Range* jest ustalana za pomocą suwaka oraz wyświetlana powyżej niego.



Rysunek 4.9 Panel ustawień wyników.

4.6.4. Informacje o autorze

W prawym dolnym rogu okna aplikacji znajduje się, widoczna na rysunku 4.10 sekcja *Author*. Zawiera ona informacje o autorze aplikacji, w tym jego imię oraz nazwisko, a także logo *Katedry Informatyki* oraz *Akademii Tarnowskiej*. Sekcja ta pełni funkcję informacyjną, wskazując twórcę aplikacji oraz jej afiliację akademicką. Dodatkowo, estetyczne umiejscowienie i czytelność tej sekcji zapewniają łatwy dostęp do informacji bez zakłócania głównej funkcjonalności interfejsu.

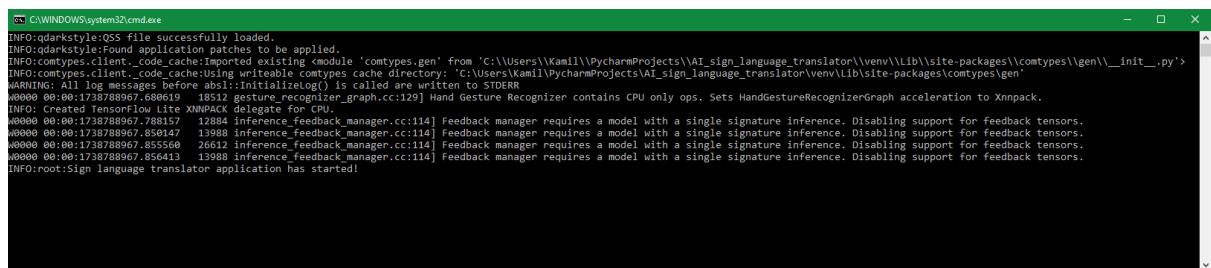


Rysunek 4.10 Sekcja informacji o autorze aplikacji.

4.6.5. Konsola diagnostyczna

System oferuje również konsolę diagnostyczną, widoczną na rysunku 4.11. Dostępność oraz wygląd konsoli zależy od środowiska uruchomieniowego. W konsoli wyświetlane są informacje, ostrzeżenia oraz błędy (na poziomie *INFO*), szczególnie te dotyczące działania bibliotek *TensorFlow Lite* oraz *MediaPipe*.

Dodatkowo raportowane są również informacje związane z obsługą wyjątków w kodzie, co ułatwia diagnozowanie potencjalnych problemów oraz poprawę stabilności aplikacji. Dzięki temu użytkownik oraz deweloper mogą śledzić działanie systemu w czasie rzeczywistym i szybko reagować na ewentualne błędy.

A screenshot of a Windows command prompt window titled "C:\WINDOWS\system32\cmd.exe". The window has a green title bar and a black background with white text. The text displays various log messages from an application, including information about loading files, applying patches, and initializing components. The logs show a sequence of events from the application's startup, including warnings about log messages and feedback tensors.

```
C:\WINDOWS\system32\cmd.exe
INFO:gdarkstyle:QSS file successfully loaded.
INFO:gdarkstyle:found application patches to be applied.
INFO:comtypes.client_code_cache:Imported existing module 'comtypes.gen' from 'C:\Users\Kamil\PycharmProjects\AI_sign_language_translator\venv\Lib\site-packages\comtypes\gen\__init__.py'
INFO:comtypes.client_code_cache:Using writeable comtypes cache directory: 'C:\Users\Kamil\PycharmProjects\AI_sign_language_translator\venv\Lib\site-packages\comtypes\gen'
WARNING: All log messages before absl:InitializeLog() is called are written to STDERR
000000 00:00:1738788967.600619 18512 gesture_recognizer_graph.cc:129] Hand Gesture Recognizer contains CPU only ops. Sets HandGestureRecognizerGraph acceleration to Xnnpack.
INFO: Created TensorFlow Lite XNNPACK delegate for CPU.
000000 00:00:1738788967.788157 12884 inference_feedback_manager.cc:114] Feedback manager requires a model with a single signature inference. Disabling support for feedback tensors.
000000 00:00:1738788967.850147 13988 inference_feedback_manager.cc:114] Feedback manager requires a model with a single signature inference. Disabling support for feedback tensors.
000000 00:00:1738788967.855500 26612 inference_feedback_manager.cc:114] Feedback manager requires a model with a single signature inference. Disabling support for feedback tensors.
000000 00:00:1738788967.856413 13988 inference_feedback_manager.cc:114] Feedback manager requires a model with a single signature inference. Disabling support for feedback tensors.
INFO:root:Sign language translator application has started!
```

Rysunek 4.11 Konsola diagnostyczna aplikacji.

5. Testy systemu

System został przetestowany pod kątem kilku kluczowych aspektów, aby zweryfikować jego poprawność działania, wydajność oraz użyteczność. W ramach testów sprawdzono kluczowe aspekty działania aplikacji, w tym poprawność rozpoznawania gestów, generowania tekstu i mowy oraz funkcjonowanie interfejsu użytkownika. Testy miały na celu ocenę stabilności aplikacji, skuteczności rozpoznawania gestów oraz ogólnej responsywności systemu. Sprawdzane gesty były prezentowane przez różne osoby w różnych warunkach, co jest widoczne na rysunkach 5.6, 5.8, 5.9 oraz 5.5.

5.1. Ewaluacja modelu

Po pomyślnym zakończeniu procesu treningu przeprowadzono ewaluację modelu, widoczną na listingu 5.1. Użyto dane z nowego zbioru testowego, który nie był używany podczas uczenia. Celem tego etapu było zweryfikowanie zdolności modelu do generalizacji na wcześniej niewidzianych danych oraz ocena dokładności klasyfikacji gestów dłoni.

Uzyskane wyniki ewaluacji przedstawiają się następująco:

- Strata (ang. *loss*) – 0.022805865854024887 (~2,3%) – wartość ta odzwierciedla błąd modelu podczas klasyfikacji danych testowych. Niska wartość straty wskazuje na dobrą jakość dopasowania modelu.
- Dokładność (ang. *accuracy*) – 0.9817684888839722 (~98,2%) – wysoka wartość dokładności świadczy o skuteczności modelu w poprawnym rozpoznawaniu gestów ze zbioru testowego.

Uzyskane wyniki potwierdzają wysoką efektywność modelu oraz jego zdolność do rozpoznawania gestów dłoni z dużą precyzją.

Listing 5.1. Ewaluacja modelu na zbiorze testowym.

```
loss, accuracy = model.evaluate(data=test_data, batch_size=16)
print(f"Test loss: {loss}, Test accuracy: {accuracy}")
```

5.2. Testy urządzeń rejestrujących

Testy urządzeń rejestrujących miały na celu ocenę, jak różne urządzenia (np. kamery internetowe, kamery wbudowane w laptopach) wpływają na jakość rozpoznawania gestów dłoni oraz wydajność i działanie systemu. Do testów wykorzystano 3 różne kamery z interfejsem *USB*, które wybrano z listy widocznej na rysunku 5.4 Na rysunkach 5.2, 5.1 oraz 5.3 widoczne są zdjęcia kamer wykorzystanych w testach. Testy te wykonano w różnych warunkach oświetleniowych oraz przy różnych rozdzielczościach obrazu, które opisano w podrozdziale 5.3.



Rysunek 5.1 Kamera internetowa Qilive.

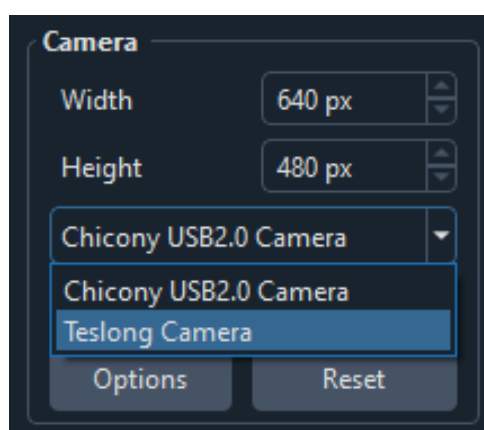


Rysunek 5.2 Kamera internetowa Chicony USB2.0.



Rysunek 5.3 Kamera internetowa Genius Eye 312

Przeprowadzone testy wykazały, że system dobrze obsługuje różne modele kamer. Wszystkie podłączone urządzenia rejestrujące wyświetlają się poprawnie na liście widocznej na rysunku 5.4 Tempo przetwarzania obrazu było ograniczone do liczby klatek obrazu na sekundę (*FPS*), które kamera jest w stanie zapewnić. Liczba tych klatek zależna była również od warunków oświetleniowych. Obraz był poprawie skalowany i konwertowany niezależnie od wybranej kamery.



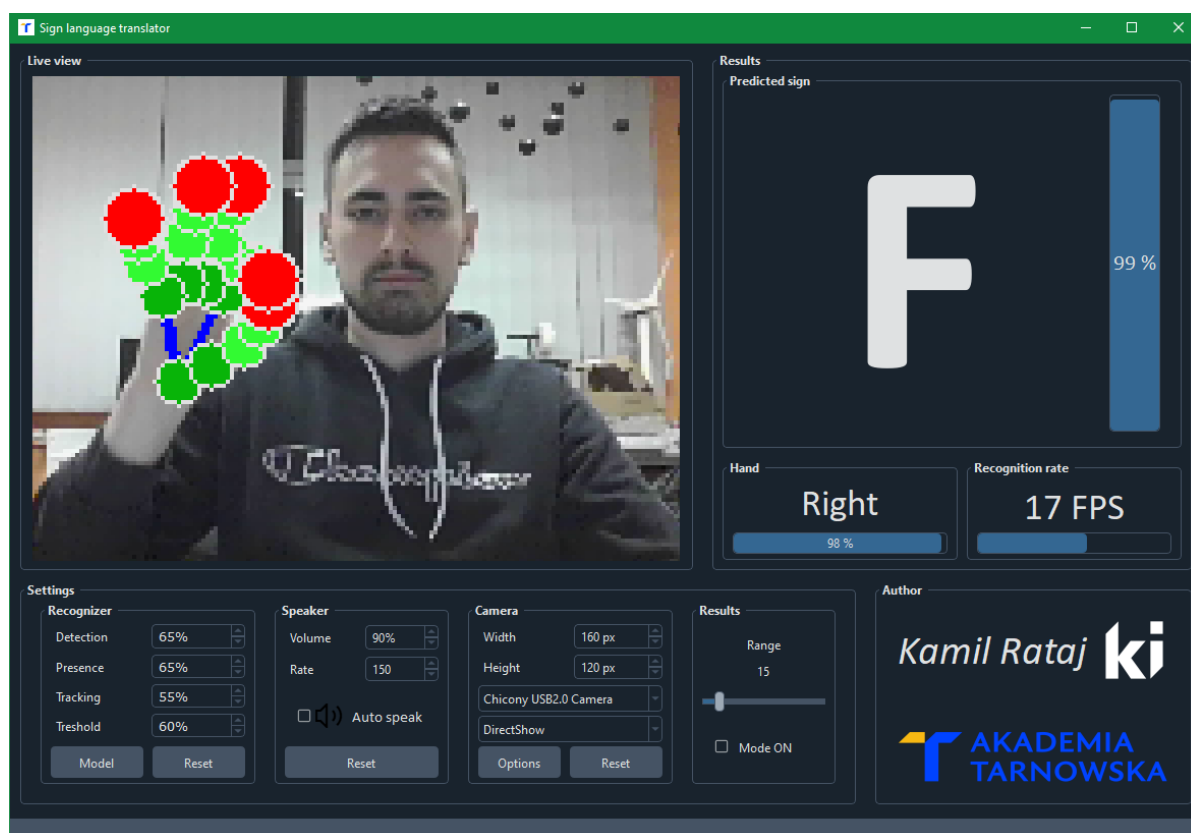
Rysunek 5.4 Lista wyboru dostępnych kamer.

5.3. Testy rozdzielczości kamery

W celu oceny wpływu rozdzielczości obrazu na skuteczność rozpoznawania gestów oraz wydajności systemu w czasie rzeczywistym przeprowadzono testy przy użyciu różnych wartości rozdzielczości. Rozdzielczości te zostały przedstawione w tabeli 5.1. Zastosowano sześć standardów formatu obrazu, od bardzo niskiej jakości (*QQVGA*), widocznej na rysunku 5.7, po wysoką rozdzielczość (*SXGA*–).

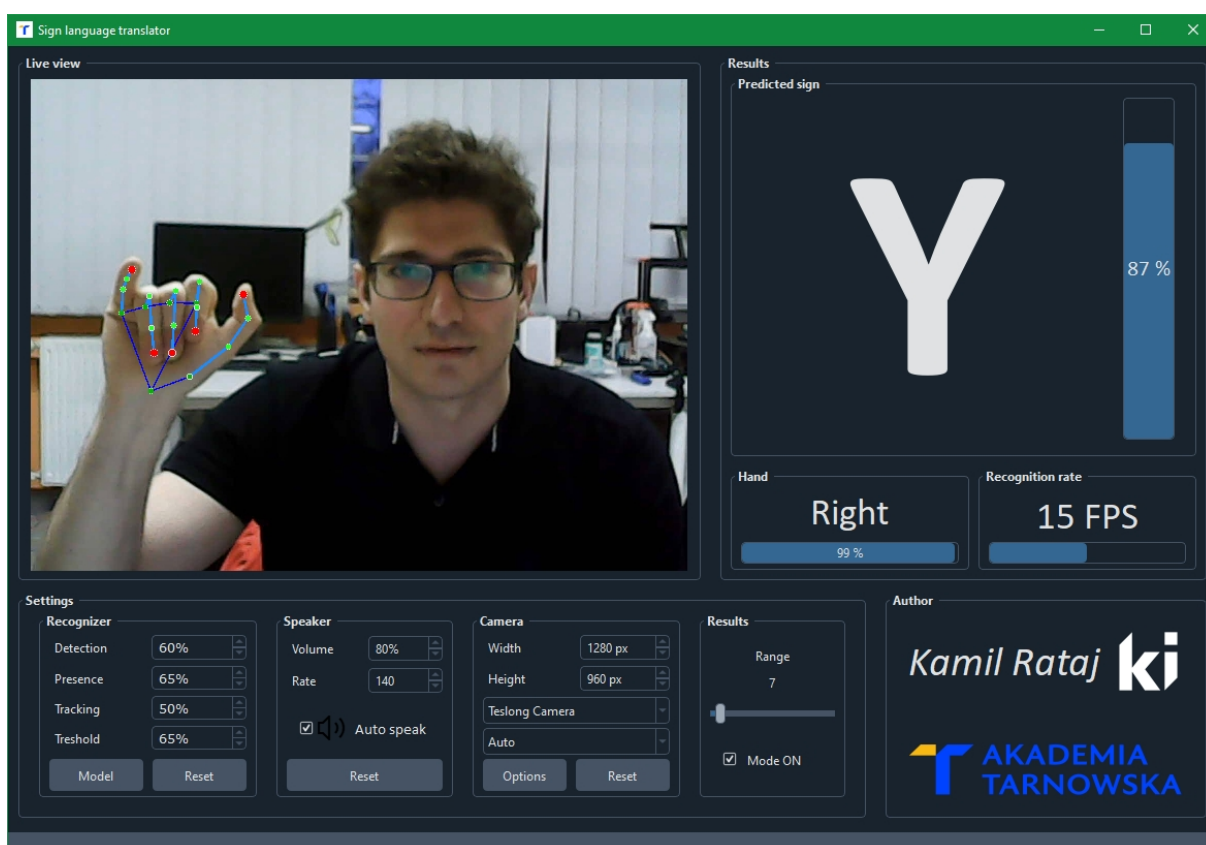
Tabela 5.1. Testowane rozdzielczości obrazu.

Szerokość [px]	Wysokość [px]	Nazwa standardu	Liczba pikseli [Mpx]
160	120	<i>QQVGA</i>	0,019
480	360	<i>360p</i>	0,173
640	480	<i>VGA</i>	0,307
800	600	<i>SVGA</i>	0,48
1280	720	<i>HD</i>	0,922
1280	960	<i>SXGA</i> –	1,229



Rysunek 5.5 Zrzut ekranu okna aplikacji z zastosowaniem rozdzielczości *QQVGA*.

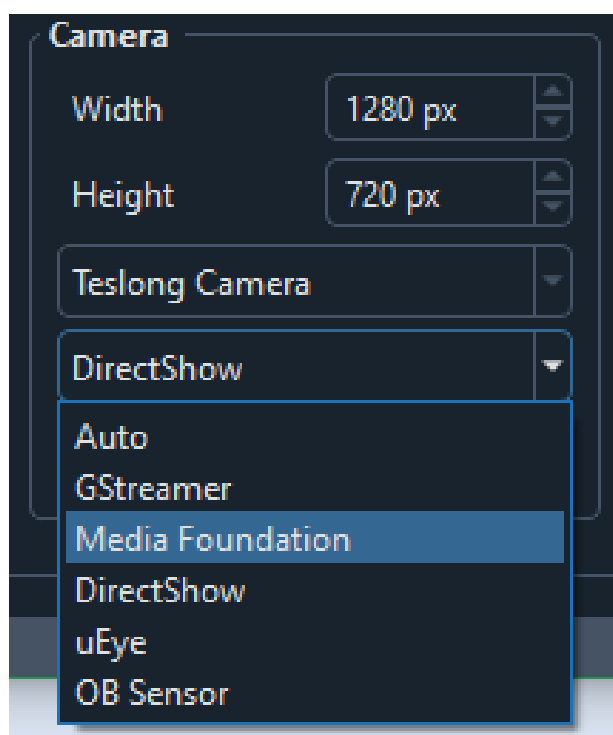
Obraz był poprawie skalowany i konwertowany niezależnie od zastosowanej rozdzielczości. Wraz ze wzrostem rozdzielczości rosła jakość rozpoznawania gestów, kosztem wydajności. W przypadku niskiej rozdzielczości, widocznej na rysunku 5.6 system nadal był w stanie rozpoznawać gesty dłoni, jednak pojawiało się dużo fałszywych rozpoznań. Ponadto wizualizacja punktów orientacyjnych dłoni przy tej rozdzielczości była nieczytelna. Najbardziej optymalną rozdzielczością obrazu okazała się 640×480 [px] (VGA).



Rysunek 5.6 Zrzut ekranu okna aplikacji z zastosowaniem rozdzielczości SXGA-.

5.4. Testy interfejsów kamery

Testowanie interfejsów kamer miało na celu ocenę ich wpływu na wydajność systemu i jakość rejestrowanego obrazu. W ramach tych testów wykorzystano różne interfejsy, takie jak np. *DirectShow*, a także inne popularne biblioteki dostępu do kamery oraz tryb automatycznego doboru interfejsu. Lista dostępnych interfejsów przedstawiona jest na rysunku 5.7 Testy wykonano na różnych komputerach.



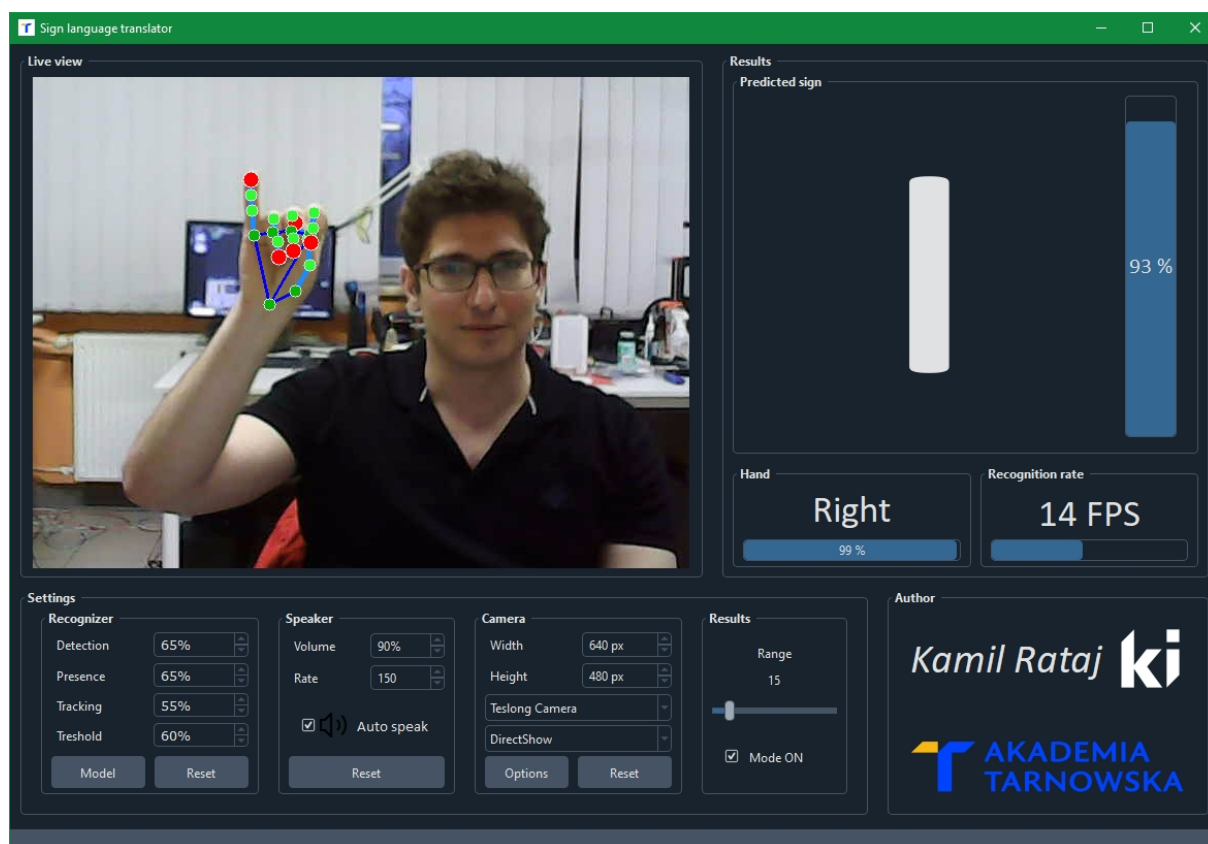
Rysunek 5.7 Lista dostępnych interfejsów kamery.

Tryb automatyczny nie w każdym przypadku dobiera najlepszy sterownik. Podczas korzystania z kamery widocznej na rysunku 5.2 najbardziej optymalnym wyborem okazał się interfejs *DirectShow*. Należy zwrócić uwagę na wybór odpowiedniego interfejsu, ponieważ nie wszystkie dostępne interfejsy obsługują każdą kamerę. W przypadku niepowodzenia inicjalizacji kamery pojawia się odpowiednia informacja o błędzie w konsoli diagnostycznej. Zmiana interfejsu kamery wpływała głównie na tempo przetwarzania oraz wydajność.

5.5. Testy funkcjonalne

Przeprowadzono szereg testów funkcjonalnych mających na celu potwierdzenie, iż system realizuje wszystkie swoje funkcje zgodnie z wymaganiami. Testy te miały na celu sprawdzenie:

- Przetwarzania obrazu i wykrywania dłoni przez system.
- Poprawności klasyfikacji gestów przez model *TensorFlow Lite*.
- Wyświetlania wyników rozpoznawania w interfejsie użytkownika.
- Prawidłowego działania syntezy mowy dla rozpoznanych znaków.
- Możliwości dostosowywania parametrów modelu i prezentacji wyników w ustawieniach aplikacji.

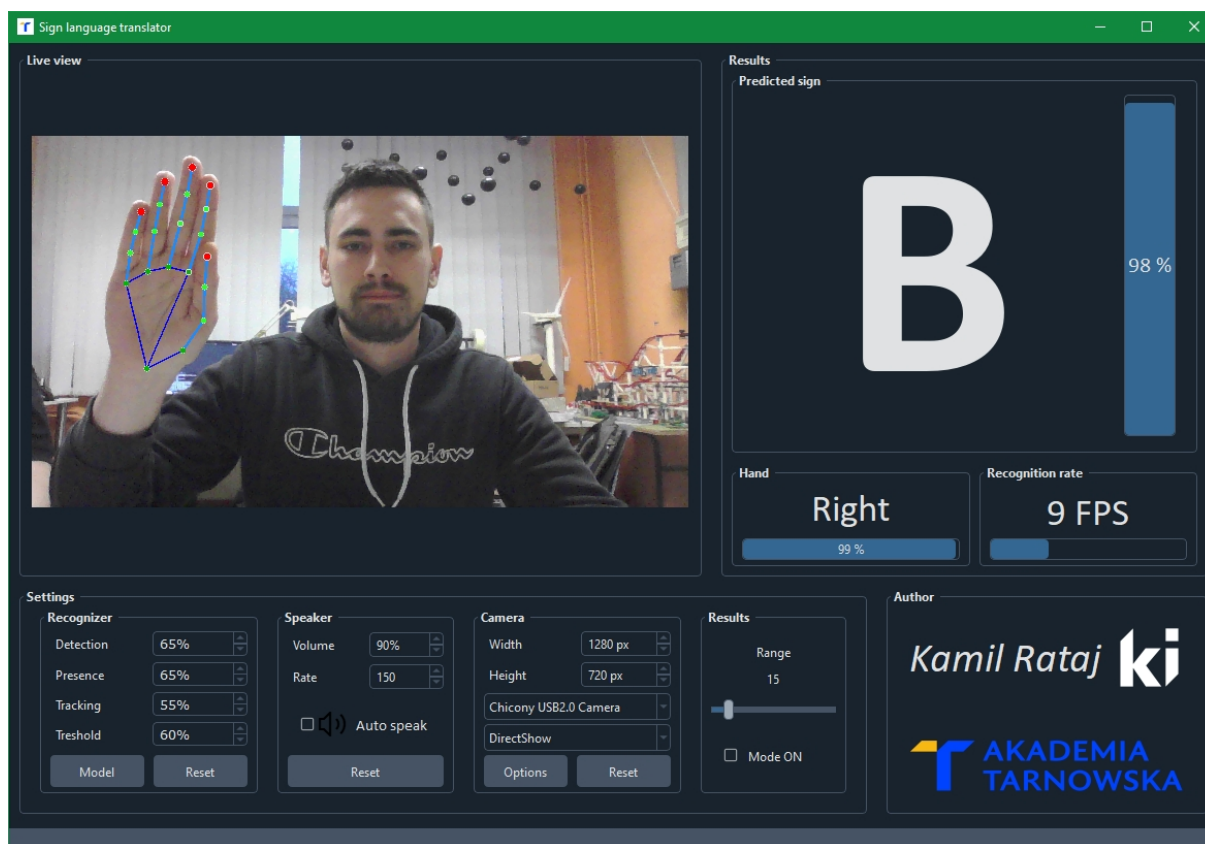


Rysunek 5.8 Zrzut ekranu aplikacji - prezentacja znaku „I”.

5.5.1. Test rozpoznawania gestów

Testy rozpoznawania gestów miały na celu ocenę skuteczności systemu w identyfikacji znaków języka migowego oraz generowaniu odpowiednich wyników. Procedura testowa obejmowała następujące kroki:

- Uruchomienie aplikacji – sprawdzono, czy system uruchamia się poprawnie oraz inicjalizuje wszystkie moduły, a także czy występują błędy z tym związane.
- Identyfikacja gestów i wyświetlanie wyników – użytkownik wykonywał określone gesty, a system analizował obraz z kamery w czasie rzeczywistym, klasyfikował gest i wyświetlał rozpoznany znak na ekranie oraz wizualizował punkty orientacyjne dłoni.
- Weryfikacja poprawności wyników – przeprowadzono testy poprawności rozpoznawania gestów, porównując rzeczywiście prezentowany znak z wynikami widocznymi na ekranie.



Rysunek 5.9 Zrzut ekranu aplikacji - prezentacja znaku „B”.

5.5.2. Test generowania tekstu i mowy

Moduł generowania tekstu i syntezy mowy był kluczowym elementem systemu, umożliwiającym użytkownikom komunikację głosową na podstawie rozpoznanych gestów. Testy tej funkcji obejmowały następujące kroki:

- Sprawdzenie generowania tekstu na podstawie gestu – po prawidłowym rozpoznaniu gestu system powinien wyświetlić odpowiadający mu tekst na ekranie. Weryfikowano zgodność wyświetlanego tekstu z oczekiwanym wynikiem.
- Synteza mowy – po wyświetleniu tekstu system uruchamiał syntezytor mowy i odczytywał rozpoznany znak. Sprawdzono, czy głos jest generowany poprawnie oraz jego użyteczność.
- Regulacja parametrów mowy – zweryfikowano działanie ustawień regulacji głośności i tempa mowy w interfejsie użytkownika. Test polegał na zmianie tych parametrów i ocenie wpływu na jakość generowanego dźwięku.

Testy modułu generowania tekstu i syntezy mowy potwierdziły poprawność wyświetlania rozpoznanych gestów w formie tekstu pisanego oraz jego konwersji na mowę. Sprawdzono zgodność generowanego tekstu z oczekiwanym wynikiem oraz jakość odtwarzanego głosu. Dodatkowo przetestowano możliwość regulacji parametrów mowy, takich jak głośność i tempo, co zapewniło większą elastyczność i dostosowanie systemu do preferencji użytkownika.

5.5.3. Test interfejsu użytkownika

Testy interfejsu użytkownika miały na celu sprawdzenie, czy wszystkie elementy graficzne aplikacji działają poprawnie i zapewniają wygodną obsługę. Przeprowadzono następujące testy:

- Sprawdzenie kompatybilności z różnymi urządzeniami – interfejs aplikacji przetestowano na różnych komputerach oraz w środowiskach o różnych rozdzielczościach ekranu, aby upewnić się, że wszystkie elementy są czytelne i prawidłowo rozmieszczone.
- Testowanie elementów sterujących – zweryfikowano działanie wszystkich przycisków, suwaków i innych kontrolki interfejsu. Sprawdzone, czy kliknięcie każdego elementu wywołuje odpowiednią akcję oraz czy zmiany ustawień są poprawnie odzwierciedlane w działaniu aplikacji.
- Ocena dostępności i responsywności – przetestowano aplikację pod kątem łatwości obsługi przez osoby o różnych poziomach zaawansowania. Sprawdzone, czy użytkownicy mogą intuicyjnie korzystać z dostępnych funkcji, a aplikacja poprawnie reaguje na zmiany ustawień i dane wejściowe.

Przeprowadzone testy interfejsu użytkownika potwierdziły poprawne działanie wszystkich elementów graficznych oraz ich dostosowanie do różnych środowisk i rozdzielczości ekranu. Zweryfikowano funkcjonalność przycisków i kontrolki, zapewniając intuicyjną obsługę i zgodność z oczekiwanym działaniem. Oceniono również dostępność i responsywność aplikacji, co pozwoliło upewnić się, że jest ona przyjazna dla użytkowników o różnych poziomach zaawansowania.

6. Podsumowanie i wnioski

Niniejsza praca inżynierska poświęcona była problemowi wykluczenia osób niesłyszących i słabosłyszących poprzez stworzenie systemu rozpoznawania i tłumaczenia amerykańskiego alfabetu migowego (*ASL*). Celem projektu było opracowanie aplikacji, która w czasie rzeczywistym rozpoznaje statyczne znaki *ASL* i tłumaczy je na tekst oraz mowę, ułatwiając komunikację i niwelując bariery.

W ramach pracy przeprowadzono dogłębną analizę literatury dotyczącej języka migowego, technik rozpoznawania obrazu i sztucznej inteligencji. Wykorzystano popularne narzędzia, takie jak język *Python*, platforma *Google Colab*, biblioteki *TensorFlow* i *OpenCV* oraz pakiet *MediaPipe Solutions*. Kluczowym elementem projektu było wytrenowanie modelu klasyfikacyjnego przy użyciu techniki uczenia transferowego. W tym celu zgromadzono i przetworzono zbiór danych zawierający zdjęcia gestów dłoni z *ASL*, pochodzący z platformy *Kaggle*. Model został zoptymalizowany pod kątem wydajności i dokładności rozpoznawania. Zaimplementowano aplikację, która przechwytuje obraz z kamery, przetwarza go, rozpoznaje gesty dłoni i tłumaczy je na tekst pisany w czasie rzeczywistym. Dodatkowo, system oferuje możliwość syntezy mowy, co umożliwia głosowe odczytywanie rozpoznanych znaków *ASL*.

Przeprowadzono szeroki zakres testów, w tym ewaluację modelu, testy urządzeń rejestrujących, testy rozdzielczości kamery, testy interfejsów oraz testy funkcjonalne. Wyniki testów wykazały, że system działa prawidłowo oraz osiąga wysoką dokładność w rozpoznawaniu statycznych znaków *ASL*. Aplikacja działa w czasie rzeczywistym, zapewniając płynną komunikację. System jest również elastyczny i pozwala na dostosowanie wielu parametrów.

W przyszłości możliwy jest dalszy rozwój systemu pod wieloma względami. Rozszerzenie systemu o rozpoznawanie dynamicznych znaków migowych, umożliwiłoby tłumaczenie całych zdań i zwrotów. Integracja z platformami mobilnymi, pozytywnie wpłynie na dostępność aplikacji. Implementacja obsługi innych języków migowych sprawi, że system będzie mógł być wykorzystywany przez osoby z różnych krajów.

Bibliografia

- [1] Grzegorz Błahut, „Obszary i formy wykluczenia osób z uszkodzonym słuchem - studium przypadku”, 2017
- [2] Olgierd Kosiba, Piotr Grenda, „Leksykon języka migowego”, Silentum, 2011
- [3] World Health Organization, „Deafness and hearing loss”, 2024, <https://www.who.int/news-room/fact-sheets/detail/deafness-and-hearing-loss/> (dostęp 01/2025 r.)
- [4] Wydawnictwo Naukowe PWN, „Internetowa encyklopedia PWN”, 2025, <https://encyklopedia.pwn.pl/> (dostęp 01/2025 r.)
- [5] NIDCD, „American Sign Language”, 2019, <https://www.nidcd.nih.gov/health/american-sign-language/> (dostęp 01/2025 r.)
- [6] Wikimedia Foundation Inc., „American Sign Language”, 2024, https://en.wikipedia.org/wiki/American_Sign_Language/ (dostęp 01/2025 r.)
- [7] <https://signsanddisplays.wordpress.com/2011/03/15/american-sign-language-alphabet-free-vectors/> (dostęp 01/2025 r.)
- [8] Adrian Kaehler, Gary Bradski, „OpenCv 3 Komputerowe rozpoznawanie obrazu w C++ przy użyciu biblioteki OpenCV”, HELION, 2018
- [9] Sebastian Raschka, Vahid Mirjalili, „Python Machine learning i deep learning Biblioteki scikit-learn i TensorFlow 2”, Helion SA, 2021
- [10] Kyle Gallatin, Chris Albon, „Uczenie maszynowe w Pythonie Receptury Od przygotowania danych do deep learningu”, Helion S.A, 2021
- [11] <https://learnopencv.com/image-classification-using-transfer-learning-in-pytorch/> (dostęp 01/2025 r.)
- [12] Python Software Foundation, „The Python Language Reference”, 2025, <https://docs.python.org/3/%20> (dostęp 01/2025 r.)
- [13] <https://www.python.org/community/logos/> (dostęp 01/2025 r.)
- [14] Google LLC, „Colaboratory Frequently Asked Questions”, 2025, <https://research.google.com/colaboratory/faq.html/> (dostęp 01/2025 r.)
- [15] <https://colab.research.google.com/> (dostęp 01/2025 r.)
- [16] JetBrains, „Pycharm Getting started”, 2025, <https://www.jetbrains.com/help/pycharm/getting-started.html/> (dostęp 01/2025 r.)
- [17] Jupyter Team, „Project Jupyter Documentation”, 2025, <https://docs.jupyter.org/en/latest/> (dostęp 01/2025 r.)
- [18] <https://www.qt.io/product/framework> (dostęp 01/2025 r.)
- [19] Qt Community, „Qt Wiki”, 2025, <https://wiki.qt.io/Main> (dostęp 01/2025 r.)
- [20] Google LLC, „XNNPACK Readme”, 2023, <https://github.com/google/XNNPACK/> (dostęp 01/2025 r.)
- [21] <https://opencv.org/media-kit/> (dostęp 01/2025 r.)
- [22] <https://www.tensorflow.org/> (dostęp 01/2025 r.)
- [23] Akash Nagaraj, „ASL Alphabet”, 2018, <https://www.kaggle.com/datasets/grassknoted/asl-alphabet/> (dostęp 01/2025 r.)
- [24] Joshua Adegoke, „A Beginner’s Guide to Kaggle for Data Science”, 2023, <https://www.makeuseof.com/beginners-guide-to-kaggle/> (dostęp 01/2025 r.)
- [25] Software Freedom Conservancy, „Git”, 2025, <https://git-scm.com/> (dostęp 01/2025 r.)
- [26] GitHub Inc., „About GitHub and Git”, , <https://docs.github.com/en/get-started/start-your-journey/about-github-and-git/> (dostęp 01/2025 r.)
- [27] The Qt Company Ltd., „Qt for Python”, 2025, <https://doc.qt.io/qtforpython-6/> (dostęp 01/2025 r.)

- [28] Colin Duquesnoy, „QdarkStyleSheet”, 2021,
<https://qdarkstylesheet.readthedocs.io/en/latest/> (dostęp 01/2025 r.)
- [29] Natesh Bhat, „pyttsx3 Readme”, 2024, <https://github.com/nateshmbhat/pyttsx3> (dostęp 01/2025 r.)
- [30] Google LLC, „MediaPipe Model Maker”, 2024,
https://ai.google.dev/edge/mediapipe/solutions/model_maker/ (dostęp 01/2025 r.)
- [31] Google LLC, „HandLandmarker (full) Model Card”, 2021,
[https://storage.googleapis.com/mediapipe-assets/Model%20Card%20Hand%20Tracking%20\(Lite_Full\)%20with%20Fairness%20Oct%202021.pdf%20](https://storage.googleapis.com/mediapipe-assets/Model%20Card%20Hand%20Tracking%20(Lite_Full)%20with%20Fairness%20Oct%202021.pdf%20) (dostęp 01/2025 r.)
- [32] Google LLC, „Hand landmarks detection guide”, 2025,
https://ai.google.dev/edge/mediapipe/solutions/vision/hand_landmarker (dostęp 01/2025 r.)
- [33] Google LLC, „HandGestureClassifier Model Card”, 2022,
https://storage.googleapis.com/mediapipe-assets/gesture_recognizer/model_card_hand_gesture_classification_with_faireness_2022.pdf%20 (dostęp 01/2025 r.)

Spis listingów

Listing 3.1. Przykładowa zawartość pliku kaggle.json.....	25
Listing 3.2. Pobranie oraz zapis obrazów używanych w treningu.....	25
Listing 3.3. Utworzenie struktury Dataset.....	26
Listing 3.4. Podział zbioru danych treningowych.....	27
Listing 3.5. Hiperparametry treningu.....	29
Listing 4.1. Skrypt startowy run_venv.bat do uruchamiania aplikacji.....	35
Listing 4.2. Konstruktor klasy CameraApp.....	37
Listing 4.3. Metoda read() klasy CameraApp.....	37
Listing 4.4. Metoda recognize_frame() klasy RecognizerApp.....	38
Listing 4.5. Metoda handle_result klasy RecognizerApp.....	42
Listing 4.6. Funkcja create_scaled_qpixmap() konwertująca i skalująca obraz wynikowy.....	44
Listing 5.1. Ewaluacja modelu na zbiorze testowym.....	52

Spis tabel

Tabela 3.1. Właściwości środowiska Colaboratory.....	24
Tabela 3.2. Opis warstw modelu.....	31
Tabela 5.1. Testowane rozdzielczości obrazu.....	55

Spis rysunków

Rysunek 1.1 Mapa występowania Amerykańskiego języka migowego. [6].....	7
Rysunek 1.2 Wykaz znaków amerykańskiego alfabetu palcowego. [7].....	8
Rysunek 1.3 Uczenie transferowe. [11].....	11
Rysunek 2.1 Logo języka Python. [13].....	12
Rysunek 2.2 Logo platformy Google Colab. [15].....	13
Rysunek 2.3 Logo środowiska Pycharm. [16].....	13
Rysunek 2.4 Logo Project Jupyter. [17].....	14
Rysunek 2.5 Schemat blokowy Qt Framework. [18].....	14
Rysunek 2.6 Diagram części składowych pakietu MediaPipe Solutions. [20].....	15
Rysunek 2.7 Logo biblioteki OpenCV. [21].....	16
Rysunek 2.8 Logo biblioteki TensorFlow. [22].....	18
Rysunek 2.9 Logo platformy Kaggle. [23].....	18
Rysunek 2.10 Logo systemu Git. [25].....	19
Rysunek 2.11 Zrzut ekranu repozytorium projektu na platformie GitHub.....	20
Rysunek 2.12 Schemat blokowy architektury środowiska Qt for Python. [19].....	21
Rysunek 2.13 Zrzut ekranu narzędzia Qt Designer.....	22
Rysunek 2.14 Logo biblioteki pytsx3. [29].....	23
Rysunek 3.1 Przykładowe obrazy ze zbioru treningowego. [23].....	24
Rysunek 3.2 Usuwanie ostatnich warstw modelu i odbudowanie z nowymi danymi. [30].....	26
Rysunek 3.3 Wykres dokładności modelu.....	32
Rysunek 3.4 Wykres funkcji straty.....	33
Rysunek 3.5 Wykres współczynnika uczenia.....	34
Rysunek 4.1 Opis punktów orientacyjnych dłoni. [32].....	40
Rysunek 4.2 Wizualizacja punktów orientacyjnych dłoni na obrazie wynikowym.....	43
Rysunek 4.3 Zrzut ekranu interfejsu użytkownika.....	45
Rysunek 4.4 Panel wynikowy interfejsu użytkownika.....	46
Rysunek 4.5 Panel ustawień modelu.....	47
Rysunek 4.6 Panel ustawień syntezy mowy.....	48
Rysunek 4.7 Okno ustawień natywnych kamery.....	49
Rysunek 4.8 Panel ustawień urządzenia rejestrującego.....	49
Rysunek 4.9 Panel ustawień wyników.....	50
Rysunek 4.10 Sekcja informacji o autorze aplikacji.....	50
Rysunek 4.11 Konsola diagnostyczna aplikacji.....	51
Rysunek 5.1 Kamera internetowa Qilive.....	53
Rysunek 5.2 Kamera internetowa Chicony USB2.0.....	53
Rysunek 5.3 Kamera internetowa Genius Eye 312.....	54
Rysunek 5.4 Lista wyboru dostępnych kamer.....	54
Rysunek 5.5 Zrzut ekranu okna aplikacji z zastosowaniem rozdzielczości QQVGA.....	55
Rysunek 5.6 Zrzut ekranu okna aplikacji z zastosowaniem rozdzielczości SXGA.....	56
Rysunek 5.7 Lista dostępnych interfejsów kamery.....	57
Rysunek 5.8 Zrzut ekranu aplikacji - prezentacja znaku „I”.....	58
Rysunek 5.9 Zrzut ekranu aplikacji - prezentacja znaku „B”.....	59